

WIL Task 1 Documentation

The Union Lab Wedding Photography & Camryn Burger Photography

Document	Client	Module	Team Number	Team Name
Task 1 - Group Documentation	Camryn Burger Photography	INSY7315	7	Algorithm Army

Project Objective : Design a secure, usable and scalable photography-management platform that prevents double bookings, automates pricing and deposits, protects client information, and supports clients, photographers and administrators from enquiry to final delivery.

INSY7315 Work-Integrated Learning | 2026

Project team: Clifford Naude (ST10258513) GROUP 2

Mikhaeel Ismail (ST10433968) GROUP 1

Siyabonga Nkomazana (ST10444685) GROUP 1

Atlegang Seboge (ST10443510)GROUP 2

Ruan Hartman (ST10453370) GROUP 1

Ntokozo Khumalo (ST10447179) GROUP 1

GitHub link for protype: https://github.com/EMGPRS/insy7315-2026-task-1-kreyo-za

Youtube link for video demo: https://youtube.com/shorts/T3Zpu-w9Q4A?si=0-_IYx4QmzVj-Yrq

9.1.3 Table of Contents

9.1.3 Table of Contents 2
9.1.3.1 List of Figures 3
9.1.3.2 List of Tables 4
9.1.4 Introduction 5
9.1.5 Requirements 5
9.1.5.1 Business Context and Problem Statement 5
9.1.5.2 Functional Requirements 5
9.1.6 User Roles 7
9.1.7 User Stories 7
9.1.7.1 UX Journey Map 8
9.1.7.2 Work Breakdown Structure 9
9.1.8 Non-functional Requirements 10
9.1.9 Analysis Artifacts 12
9.1.9.1 Domain Class Diagram 12
9.1.9.2 Design Class Diagram 13
9.1.10 Implementation 14
9.1.10.1 Critical Sequence Diagrams 14
9.1.10.2 State Diagrams 15
9.1.10.3 Deployment Plan 15
9.1.11 Data Schema 16
9.1.11.1 Technology Choice and Conceptual ERD 16
Fully Excusable Database schema 17
9.1.11.2 Representative Firestore JSON Schemas 17
9.1.12 Architecture Artifacts 18
9.1.12.1 Design Patterns 18
9.1.12.2 Architecture Patterns 21
9.1.12.2.1 Pattern selection and justification 21
9.1.12.2.2 Implemented website traceability 22
9.1.12.2.3 Logical component responsibilities 24
9.1.12.2.4 Design-pattern integration 24
9.1.12.2.5 Critical booking interaction 24
9.1.12.2.6 Evolution and architecture acceptance tests 25
9.1.12.3 Cloud Architecture 26
9.1.12.3.1 Service selection and justification 26
9.1.12.3.2 Network and protocol flow 27
9.1.12.3.3 Availability, scaling and recovery 28
9.1.12.3.4 Assumptions and decisions to confirm 28
9.1.13 Security 28

9.1.13.1 Assets, trust boundaries and principles 28
9.1.13.2 Prototype security delta....29
9.1.13.3 Comprehensive security risk register....30
9.1.13.4 XSS prevention in depth 32
9.1.13.5 Data protection, privacy and retention 32
9.1.13.6 Security operations and incident response 33
9.1.14 DevOps....34
9.1.14.1 Version control, branching and collaboration....34
9.1.14.2 GitHub Actions Pipeline 34
9.1.14.3 Test strategy and quality gates 36
9.1.14.4 Executable workflow and repository contract....36
9.1.14.5 Deployment, database and rollback procedure 36
9.1.14.6 DevOps metrics and ownership 37
Appendix A - Executable GitHub Actions workflow 37
9.1.15 Running Costs 38
9.1.15.1 Two-year Projection 38
9.1.16 Change Management 39
9.1.17 Appendices 42
9.1.17.1 Declaration of Authenticity 42
Appendix A-Executable GitHub Actions Workflow....43
9.1.17.3 Required GitHub Environment Configuration....47
References....58

9.1.3.1 List of Figures

Figure 1. Implementation data model linking Booking, CustomerInfo, LocationInfo, Photographer, Service, PackageOption and DashboardStats. 6
Figure 2. Client, photographer and administrator UX journey map. 8
Figure 3. Work breakdown structure, part 1 of 3. 9
Figure 4. Work breakdown structure, part 2 of 3. 10
Figure 5. Work breakdown structure, part 3 of 3. 10
Figure 6. Domain UML showing customers, bookings, photographers, services, payments and add-ons....12
Figure 7. Design UML showing implementation classes, attributes, operations, access specifiers and relationships. 13
Figure 8. Weather postponement and editing-deadline interaction sequence. 14
Figure 9. Cancellation and refund decision sequence aligned to the business terms. 14
Figure 10. Booking lifecycle states and permitted transitions. 15
Figure 11. Photograph workflow from upload through editing and delivery. 15
Figure 12. Conceptual entity-relationship model used to derive Firestore collections and references. 16
Figure 13. UML design-pattern realisation for the proposed system....21
Figure 14. System architecture showing modules, layers, adapters and dependency direction. 22
Figure 15. Booking hold, hosted deposit and idempotent confirmation sequence. 25
Figure 16. Firebase and Google Cloud trust boundaries, services and network flow. 27
Figure 17. GitHub branching and gated CI/CD workflow. 35

9.1.3.2 List of Tables

Table 1. User roles and authorisation boundaries. 7
Table 2. User stories and acceptance summaries. 7
Table 3. Measurable non-functional requirements and verification approach. 10
Table 4. Firestore collections and integrity rules. 17
Table 5. Architecture pattern selection and trade-offs. 22
Table 6. Website prototype traceability to the production architecture. 23
Table 7. Logical component responsibilities. 24
Table 8. Design-pattern integration. 24
Table 9. Cloud service selection and security justification. 26
Table 10. Availability, scaling and recovery controls. 28
Table 11. Prototype security gaps and required production controls. 29
Table 12. Security risk register and mitigation strategies. 30
Table 13. Data protection and retention decisions. 32
Table 14. Version-control rules and supporting evidence. 34
Table 15. Repository evidence and CI/CD alignment. 34
Table 16. GitHub Actions stages, gates and outputs. 35
Table 17. Test strategy and pass rules. 36
Table 18. DevOps metrics and targets. 37
Table 19. Estimated running costs by user scale. 38
Table 20. Two-year running-cost projection. 38
Table 21. Recurring cost components. 39
Table 22. Projected storage growth. 39
Table 23. Change-management activities and timeline. 40
Table 24. Stakeholder communication plan. 40
Table 25. Change implementation roadmap. 40
Table 26. Training plan. 41
Table 27. Support plan. 41
Table 28. Change-management success metrics. 41
Table 29. Change-management risks and mitigations. 42
Table 30. Declaration of Authenticity signatures. 43
Table 31. GitHub environment configuration. 47

9.1.4 Introduction

This document presents management with the requirements, analysis, design, implementation, data, architecture, security, DevOps, cost and change-management plan for a unified photography booking and operations platform. This is a project for the Union Lab Wedding Photography & Camryn Burger Photography company. The business outgrew their current booking system using WhatsUp. The solution is an integrated online booking system with certain automations to improve on the company's efficiency and reduce human errors where possible. The system solves problems like double booking, manual quotations, contacting customers and deadline management. We provide a strong, secure, maintainable, scalable and cost-friendly system for this company. We will go through the architecture of these systems within this document and show the overall blueprint of the project. We had chosen certain technologies and architectural frameworks for this project to optimize it for this project that works mainly with images. The recommended solution gives clients real-time availability, dynamic quotations, hosted deposit payment, reminders and gallery access; gives photographers a controlled schedule and editing workflow; and gives administrators package, user, assignment, reporting and content controls. The design follows the supplied prototype while hardening it with server-side business rules, Firebase/Google Cloud managed services, explicit trust boundaries and a gated GitHub Actions pipeline

9.1.5 Requirements

9.1.5.1 Business Context and Problem Statement

- **Growth and concurrency.** The business grew from a small operation to more than 15 bookings per week; the proposal reports 50+ clients per month and plans for more photographers (The Union Lab Wedding Photography & Camryn Burger Photography, n.d.; INSY7315 WIL Group, 2026a).
- **Current failure modes.** WhatsApp, calls and social-media DMs create double bookings, forgotten shoots/editing deadlines, fragmented records, manual price errors and difficult payment follow-up (The Union Lab Wedding Photography & Camryn Burger Photography, n.d.).
- **Commercial rules.** A 50% deposit secures the date; the balance is due before the shoot. One non-weather/emergency postponement is allowed; a second attracts 25%. Cancellation within five working days attracts 100% or forfeits the deposit (Camryn Burger Photography, n.d.).
- **Pricing and delivery.** Travel beyond 20 km from Krugersdorp is R5/km; extra images cost R5 each; agreed images are normally due within 14 working days; images are retained for three months (Camryn Burger Photography, n.d.).
- **Required capabilities.** Real-time availability, dynamic pricing, hosted payment, photographer/admin dashboards, automated reminders, portfolio/social links, invoices, terms and conditions, analytics and cloud backup (INSY7315 WIL Group, 2026a).

9.1.5.2 Functional Requirements

9.1.5.3 Technology Sections

We had decided to use Firebase for Authentication and Store (FireStore). This is a NoSQL database. The reason for Firebase Authentication is due to its capability of storing user accounts and verifying which the user logged into the website is part of its data. This gives us improved security and high maintainability since the third party (firebase) manages the account securities and email notifications. Otherwise, we would have to manual implement our own notification service into the project, which would be an extra cost and unnecessary complex code. The use of FireStore/Firebase Database is still up for discussion, but we believe that firestore is better for giving real-time data while the database is better for simplicity. Due to the nature of the project, Firebase database will be better since real time data isn't necessary. This reduces complexity and allows for high maintainability. The reason for NoSQL is due to the flexibility of the project; we can include uploads and downloads using NoSQL, if necessary, in the later feature. We chose this instead of SQL due to the scalability of the technology, giving us flexibility in the later feature to make changes if required. We had also decided to create our own API as this API will be between the server and the client. The API talks to firebase and the website. This allows for a better architecture, so that if we need to change database and authentication

services, we can easily do it without effecting the frontend. We used Next.js for the backend with global CSS styling for the rest of the pages, (LaFalce, 2025). Next.js is a full-stack react framework that offers us service-side rendering and API routes. We chose Next.js for static and dynamic rendering for the front and backend in one framework. It provides fast page loading and performance which is important for this project as images take time to load with image optimization for reducing load times using automatic lazy loading and resizing.

The diagrams and schemes were made using these tools, ((JSON tools, 2026), (Json tools, 2026), SmartDraw.IO). We go around the booking class. This in the relationship diagram means the **Many bookings** can contain **one**: **Customers** (CustomerInfo), **Location** (LocationInfo), and **Photographer**. We also have **booking** contain **Many** services but only **one service** belongs to a booking. **Many Services** can have **one PackageOption**. The schemes are all objects are stored with data in a JSON document. This is more of a blueprint of how the object looks and what it should contain. Anyone can access the website as that is public; this is why we didn't create a user account, only a customer, photographer and admin account. The database only stores necessary information about these accounts.

Firebase Authentication and a Firebase NoSQL data store, placed an API between the browser and Firebase, and modelled the website objects as JSON documents. The implementation model below complements the conceptual ERD in Figure 8. The seven embedded schema files have been extracted and reproduced verbatim as readable JSON rather than inaccessible file icons.

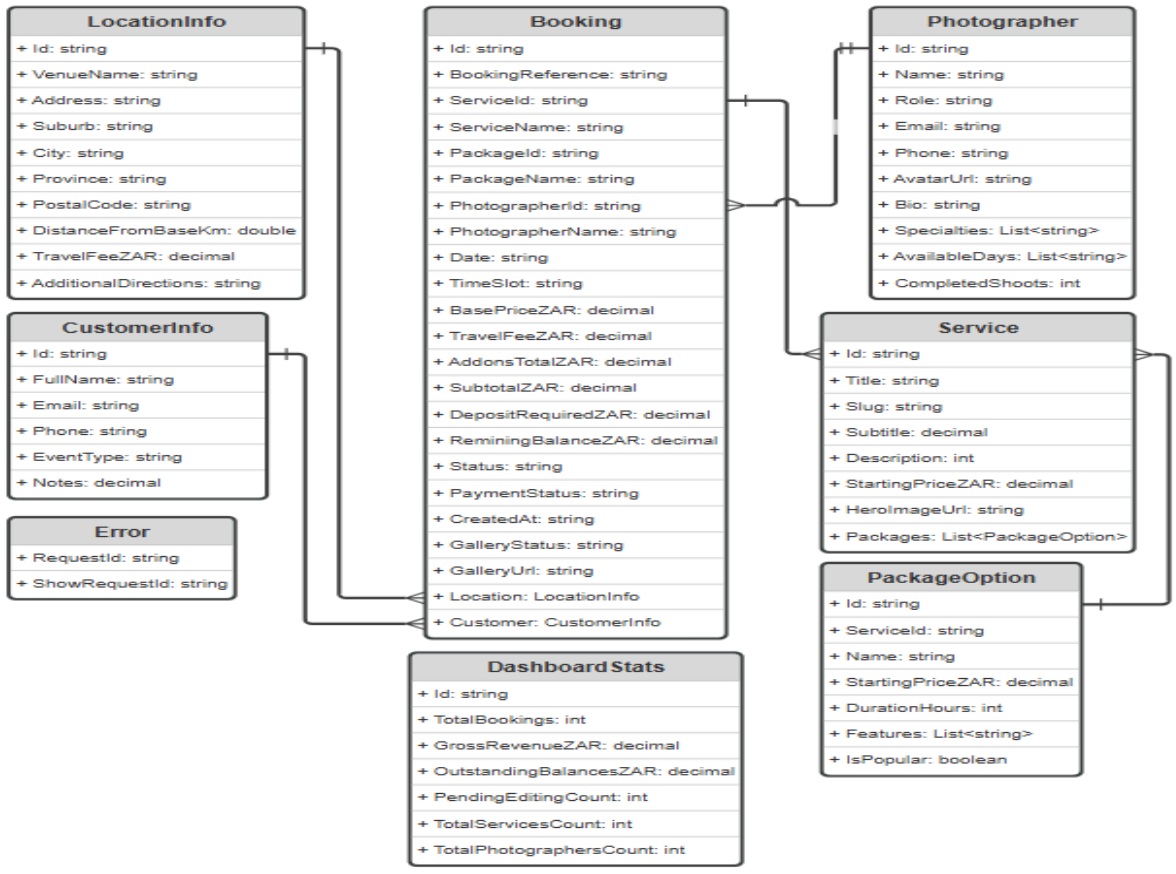


Figure 1. Implementation data model linking Booking, CustomerInfo, LocationInfo, Photographer, Service, PackageOption and DashboardStats.

9.1.6 User Roles

Table 1. User roles and authorisation boundaries.

Role	Primary responsibilities	Authorization boundary
Client / Customer	Manage own profile; browse packages; request availability; book and pay; reschedule/cancel subject to terms; receive reminders; access own galleries.	May access only their own bookings, invoices, payments and private gallery links.
Photographer	Maintain availability; view assigned bookings; update shoot/editing status; upload approved deliverables; receive deadline alerts.	May access assigned work and permitted client contact data, but may not administer pricing, roles or unrelated bookings.
Administrator	Manage services, packages, users, photographer assignment, exceptions, website/gallery content, reports and audit review.	Requires MFA and least-privilege administrative claims; sensitive actions are logged.

9.1.7 User Stories

Table 2. User stories and acceptance summaries.

ID / actor	User story	Acceptance summary
C1 Client	As a client, I want to register and maintain my details so that bookings and communication are accurate.	Validated account; unique email; own-profile access only.
C2 Client	As a client, I want to browse packages and live availability so that I can select a suitable shoot.	Only available photographer/time combinations are offered.
C3 Client	As a client, I want an automatic quotation so that I understand the full price before committing.	Package, travel and extras are itemized and recalculated server-side.
C4 Client	As a client, I want to pay the required deposit online so that my date is confirmed immediately.	Hosted payment; verified webhook; one receipt; status Confirmed.
C5 Client	As a client, I want reminders and status updates so that I do not miss important actions.	Preferred channel, delivery status and retry evidence recorded.
P1 Photographer	As a photographer, I want a calendar of assigned shoots so that I can manage my workload.	Day/week views show only authorized assignments and conflicts.
P2 Photographer	As a photographer, I want to update shoot and editing progress so that delivery deadlines remain visible.	Only valid state transitions are accepted and audited.
P3 Photographer	As a photographer, I want to upload completed photographs securely so that clients can access approved work.	Private storage; metadata validation; short-lived access links.
A1 Administrator	As an administrator, I want to manage packages, users and assignments so that operations remain accurate.	MFA, role check, validation and audit log required.
A2 Administrator	As an administrator, I want reports on bookings, revenue and outstanding balances so that management decisions use consistent data.	Date filters, totals reconciled to payment records and export control.

9.1.7.1 UX Journey Map

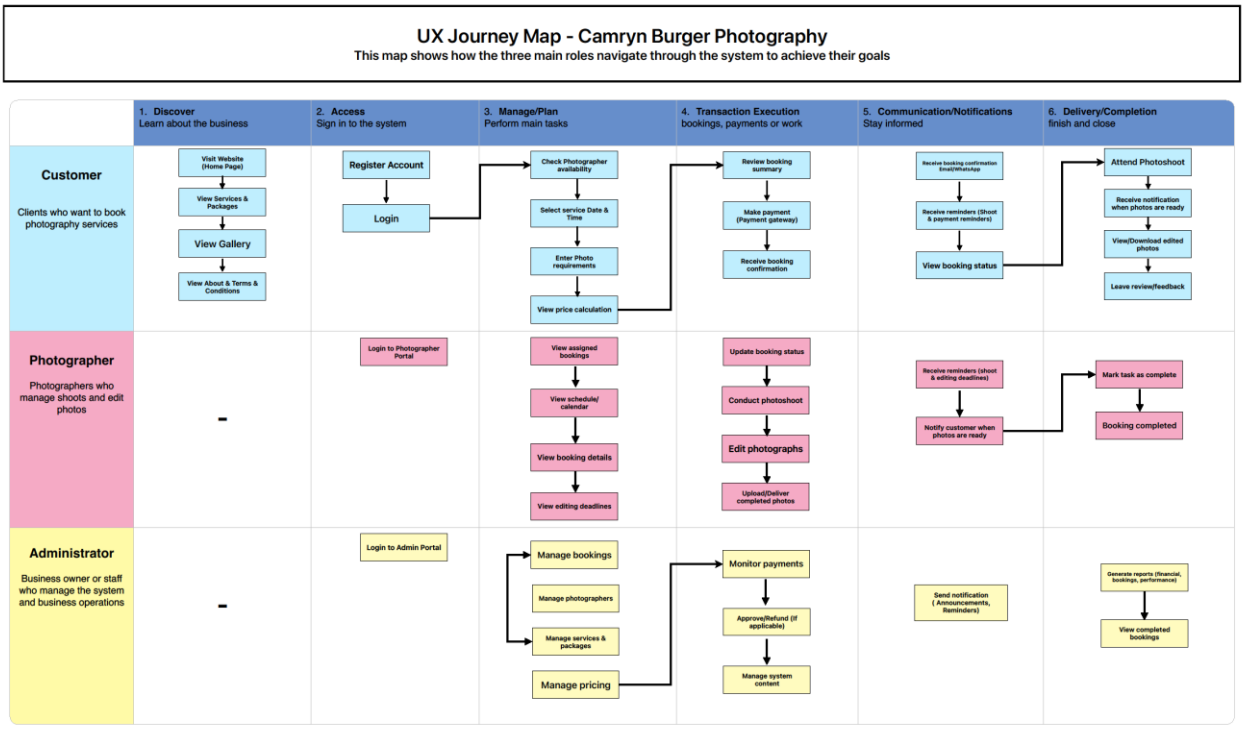


Figure 2. Client, photographer and administrator UX journey map.

The journey connects discovery, service selection, availability, quotation, terms acceptance, hosted payment, confirmation and post-shoot delivery. Failure paths return a clear recovery action: unavailable slots prompt alternatives; payment failure preserves the unconfirmed hold for a bounded period; and notification failure is retried without repeating the booking transaction.

9.1.7.2 Work Breakdown Structure

ID	Task Name	Description	Duration	Predecessor(s)	Responsible Team Member(s)	Resources
1.0	**Project Planning**	Establish the project scope, objectives, team responsibilities and initial development plan.	2 days	None	All Members	Module Manual, Google Meet, Microsoft Word
1.1	Client Requirements Analysis	Analyse the client's business processes, problems and required system functionality ansd gather neccesary requirements.	2 days	1.0	Ruan Hartman	Client brief, Google Meet, Microsoft Word
2.0	**Research and Analysis**	Investigate the photography booking environment and analyse existing problems and potential solutions.	3 days	1.1	All Members	Internet, academic sources, Google Meet
2.2	User Analysis	Analyse the needs, expectations and activities of customers and photographers.	2 days	1.1	All Members	Client brief, Google Meet, Microsoft Word
3.0	**System Requirements and Modelling**	Translate the gathered requirements into models and structured system specifications.	5 days	2.2	Ruan Hartman, Siyabonga Nkomazana	Draw.io,Freeform, Microsoft Word
3.1	User Roles and User Stories	Identify system roles and develop user stories for each role.	2 days	2.2	Siyabonga Nkomazana	Microsoft Word, Module Manual requirements documentation
3.2	UX Journey Mapping	Map the navigation and user experience through the system for the identified user roles.	2 days	3.1	Siyabonga Nkomazana	Freeform
3.3	UML Sequence Diagram	Identify and model the interactions between users and the system.	2 days	3.1	Ruan Hartman	Draw.io
3.4	Domain Modelling	Identify the main system entities, attributes and relationships.	2 days	3.1	Ruan Hartman	Draw.io
3.5	Non-functional Requirements	Define requirements relating to security, usability, performance, reliability and other quality attributes.	2 days	1.1	Siyabonga Nkomazana	Client requirements, Microsoft Word
4.0	**System and Database Design**	Design the technical structure and visual interface of the proposed system.	5 days	3.4	Mikhaeel Ismail, Atlegang Seboge	Visual Studio, SQL Server, Draw.io, Figma
4.1	Database Design	Design the database structure required to store users, bookings, packages, payments and related information.	3 days	3.4	Mikhaeel Ismail	Firebase Database
4.3	UI/UX Design	Design the website's layouts, navigation and visual interface.	4 days	3.2	Mikhaeel Ismail	Figma
5.0	**Frontend Development**	Develop the customer-facing website and user interface.	7 days	4.3	Mikhaeel Ismail	React/Next.js
5.1	Homepage and Information Pages	Develop the Home, About, Terms and Conditions and social media sections.	3 days	4.3	Mikhaeel Ismail	React/Next.js
5.2	Gallery	Develop the photography gallery showcasing the business's work.	2 days	5.1	Mikhaeel Ismail	React/Next.js, image assets
5.3	Customer Booking Interface	Develop the interface through which customers select services, dates, locations and packages.	4 days	5.1	Atlegang Seboge	React/Next.js

Figure 3. Work breakdown structure, part 1 of 3.

6.0	**Backend Development**	Implement the business logic and system functionality.	10 days	4.1, 4.2	Clifford Naude, Atlegang Seboge, Siyabonga Nkomazana, Ruan Hartman, Ntokozo Khumalo	Visual Studio, C#, .NET, database
6.1	User Authentication	Implement customer and photographer authentication and role-based access.	3 days	4.1	Clifford Naude	Visual Studio, .NET, database
6.2	Booking Management	Implement booking creation, modification and availability management.	5 days	6.1	Atlegang Seboge	Visual Studio, C#, database
6.3	Package and Price Calculation	Implement package selection, additional photographs and automatic price calculations.	4 days	6.2	Siyabonga Nkomazana	C#, database
6.4	Payment Functionality	Implement deposit and outstanding payment tracking/payment gateway functionality.	4 days	6.3	Siyabonga Nkomazana	Yoco Payment API/gateway, C#, database
6.5	Photographer Dashboard	Implement booking, payment and editing-deadline information for photographers.	4 days	6.2	Ruan Hartman	Visual Studio, C#, database
6.6	Reminder System	Implement reminders for bookings, payments and photography editing deadlines.	3 days	6.5	Ntokozo Khumalo	Email/notification service, C#
7.0	**Integration**	Connect the frontend, backend, database and external services.	4 days	5.3, 6.6	Mikhaeel Ismail, Siyabonga Nkomazana, Ntokozo Khumalo	Visual Studio, database, APIs
7.1	Database Integration	Connect system functionality to the database.	2 days	6.2	Mikhaeel Ismail	SQL Server, Entity Framework
7.2	Payment Integration	Connect the payment functionality to the selected payment service.	2 days	6.4	Siyabonga Nkomazana	Payment gateway/API (Yoco)
7.3	Notification Integration	Connect the reminder functionality to the selected notification service.	2 days	6.6	Ntokozo Khumalo	Email/notification API
8.0	**Testing and Quality Assurance**	Test the completed system against its functional and non-functional requirements.	5 days	7.3	All Members	Test cases, browsers, devices
8.1	Functional Testing	Test booking, package selection, pricing, payment and dashboard functionality.	3 days	7.1	All Members	Test cases, application
8.2	Usability Testing	Evaluate navigation, interface consistency and ease of use.	2 days	7.3	All Members	Test users, application
8.3	Security Testing	Test authentication, authorisation and protection of user information.	2 days	7.1	All Members	Application, test data
8.4	Bug Fixing	Resolve issues identified during testing and retest affected functionality.	3 days	8.1, 8.2, 8.3	All Members	Visual Studio, test results
9.0	**Documentation and Finalisation**	Complete the project documentation and prepare the final system.	5 days	8.4	All Members	Microsoft Word, screenshots, diagrams
9.1	Documentation Completion	Compile requirements, models, diagrams, testing evidence and project documentation.	3 days	8.4	All Members	Microsoft Word, Draw.io,Figma
9.2	Final System Review	Review the system against the client's requirements and identify outstanding issues.	1 day	9.1	All Members	Requirements specification, Google Meet

Figure 4. Work breakdown structure, part 2 of 3.

9.3	Final Presentation Preparation	Prepare the demonstration and presentation of the completed system.	2 days	9.2	All Members	PowerPoint, completed system
9.4	Final Submission	Submit the completed documentation and system according to the module requirements.	1 day	9.3	All Members	Final report, source code, ARC submission platform

Figure 5. Work breakdown structure, part 3 of 3.

9.1.8 Non-functional Requirements

These requirements describe how the system performs, rather than what it does. (Sommerville, 2016)

Table 3. Measurable non-functional requirements and verification approach.

ID / quality	Measurable requirement	Design and verification approach
NFR-01 Performance	•The system should load its pages within 3 second under normal network conditions. •Booking confirmations should be confirmed within 5 seconds •The system should be capable of supporting at least 100 concurrent users without performance degradation	CDN/static export, indexed queries, load test at 100 concurrent users and trace p95 latency.
NFR-02 Security	•All user accounts should require secure authentication •Customer passwords should be encrypted before storage	SAST, dependency/secret scans, DAST, rules tests and penetration-test evidence.

ID / quality	Measurable requirement	Design and verification approach
	•Online payments should use secure HTTPS encryption •Only authorised administrators may manage bookings, photographers and reports	
NFR-03 Availability	•The system must be available 24 hours a day, 7 days a week, except during scheduled maintenance	Managed services, health probes, monitoring, bounded retries and graceful notification degradation.
NFR-04 Reliability	•The system should automatically save booking information and prevent duplicate bookings •The booking and payment information should be backed up daily •The system should be capable of recovering from unexpected failures without data loss	Firestore transaction/contention tests, idempotency tests, PITR/backups and quarterly restore exercise.
NFR-05 Usability	•The system should provide a simple and responsive interface •Users shall be able to complete a booking without technical assistance (ideally within 5 minutes) •The navigation shall remain constant across all pages •The system should function correctly on desktops, tablets and mobile devices	Moderated task testing with at least five representative users and accessibility checks.
NFR-06 Scalability	•The system should support the future expansion of the organisation to multiple photographers across South Africa without requiring major redesign •The database shall accommodate increasing customer records without affecting the performance	Stateless API autoscaling, Firestore slot-document strategy, storage CDN and staged load profiles.
NFR-07 Maintainability	•The source code should be modular and well documented to support future development •Future developers should be able to add new services with minimal code changes	Architecture tests, code review, CI/CD evidence, ADRs and operational runbooks.
NFR-08 Compatibility	•The application will function correctly on the latest versions of Chrome, Edge, Safari and Firefox. •The website will be responsive across different screen sizes on desktops, tablets and mobile phones	Browser/device matrix executed for release candidates.
NFR-09 Legal/privacy	•Customer information will comply with the Protection of Personal information Act (POPIA) •Customers must accept the Terms and Conditions before confirming a booking •Payment processing shall comply with the security standards of the payment gateway	Privacy review, retention jobs, access/deletion workflow and auditable approvals.

9.1.9 Analysis Artifacts

9.1.9.1 Domain Class Diagram

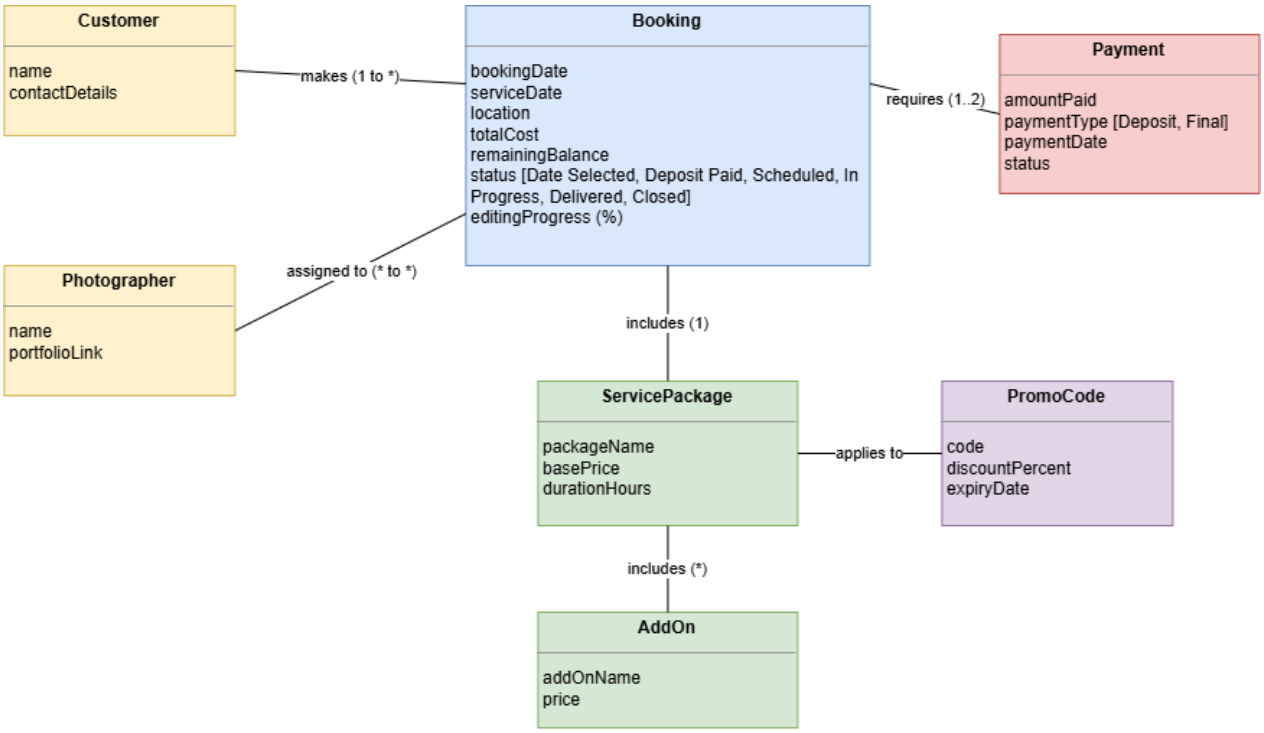


Figure 6. Domain UML showing customers, bookings, photographers, services, payments and add-ons.

The domain centres on Booking: a client creates bookings; a photographer receives bookings and owns availability; each booking selects a service/package and produces payments and an invoice; completed work is exposed through controlled gallery/media records. Cardinalities in the diagram establish ownership, while business invariants such as a 50% deposit and non-overlapping photographer slots remain aggregate rules rather than UI assumptions.

9.1.9.2 Design Class Diagram

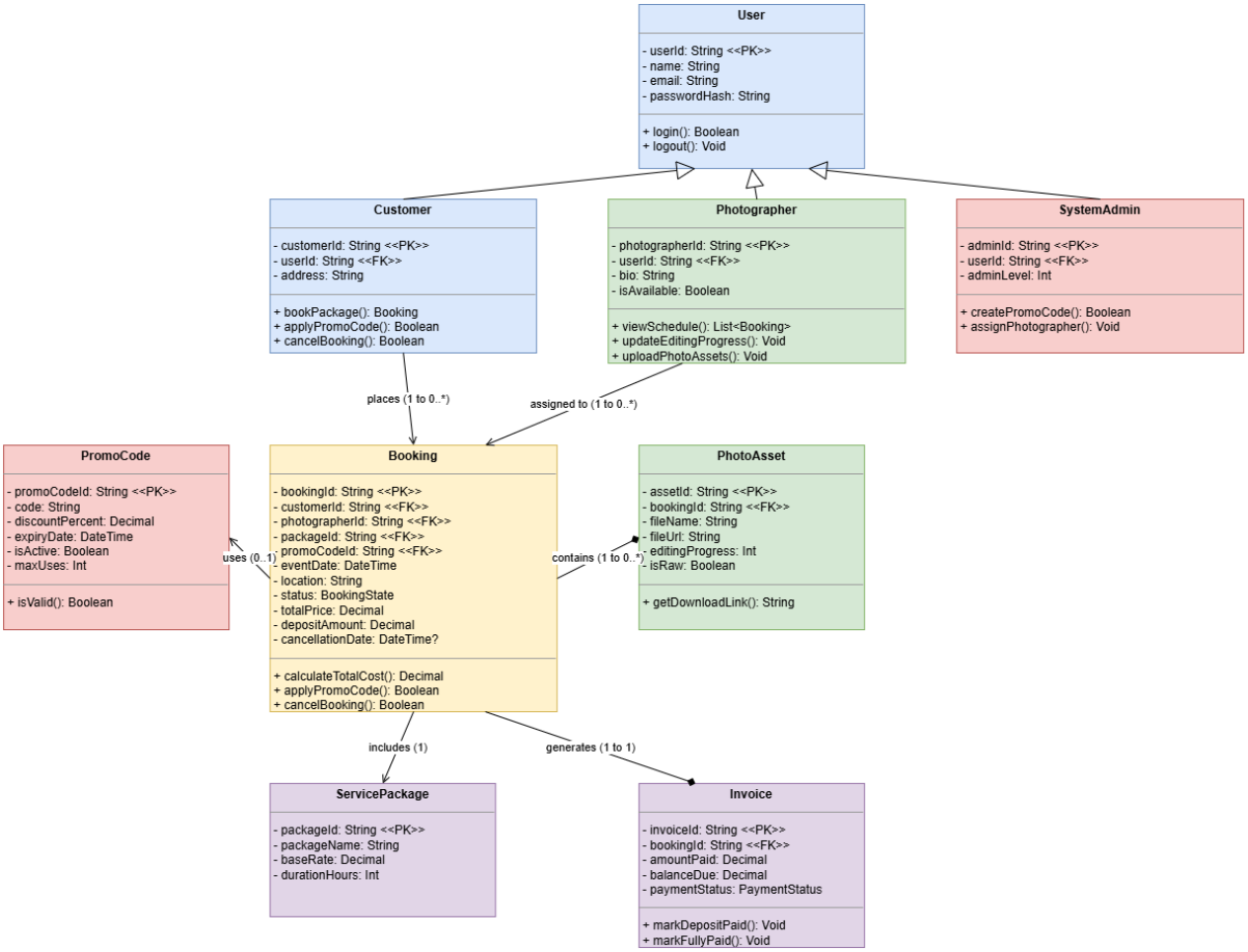


Figure 7. Design UML showing implementation classes, attributes, operations, access specifiers and relationships.

The design model translates domain concepts into controllers/services/repositories and explicit operations. In implementation, repository contracts isolate Firestore, service classes enforce use cases and controllers expose validated DTOs. Document identifiers act as logical primary keys; referenced IDs and deterministic slot documents express relationships without incorrectly treating Firestore as a relational SQL database.

9.1.10 Implementation

9.1.10.1 Critical Sequence Diagrams

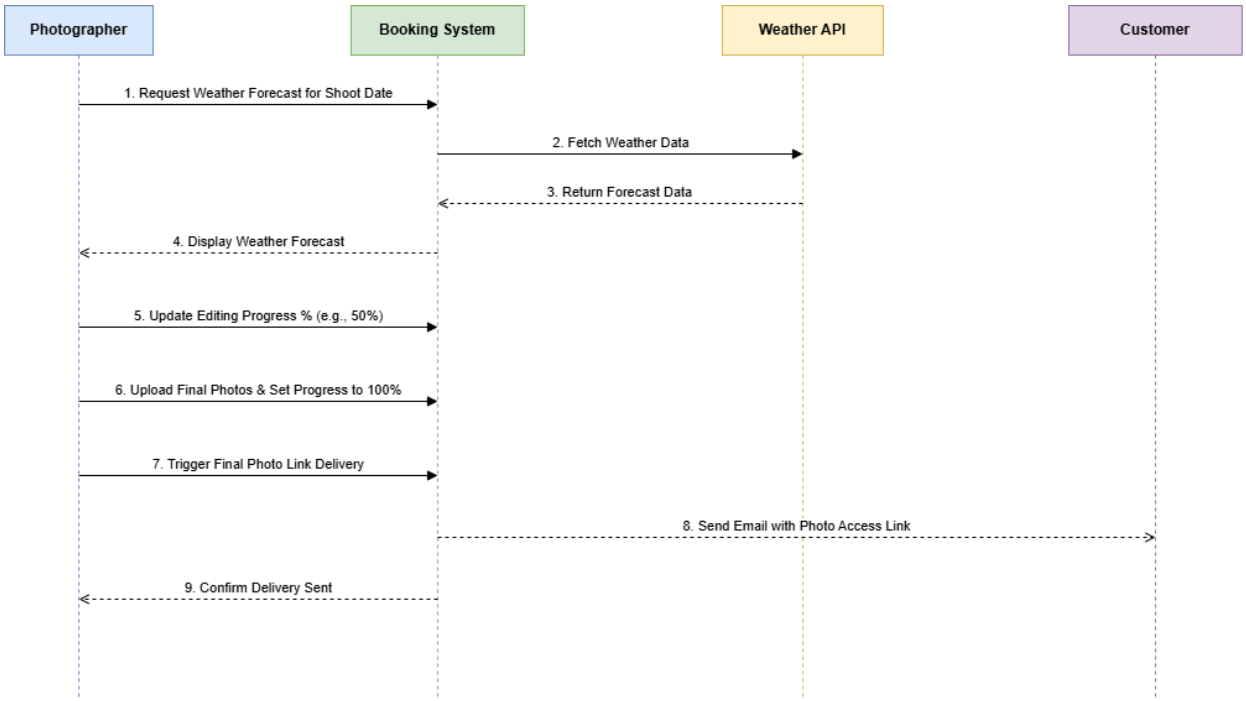


Figure 8. Weather postponement and editing-deadline interaction sequence.

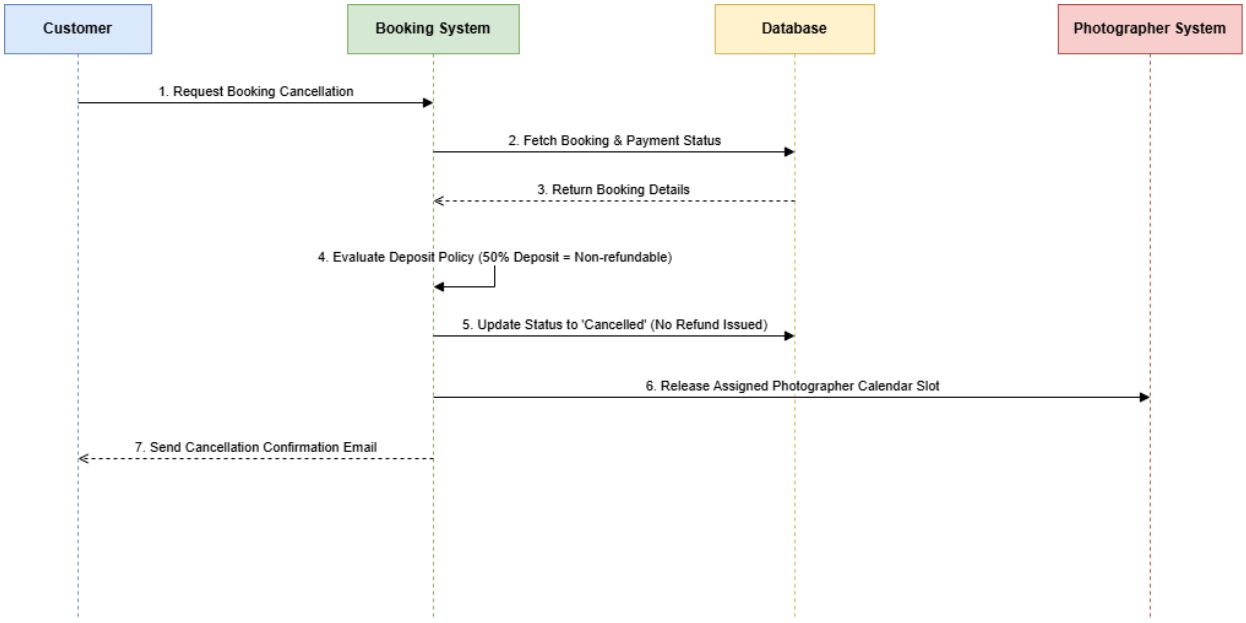


Figure 9. Cancellation and refund decision sequence aligned to the business terms.

The booking/deposit sequence is presented in Figure 11 because it includes the final 50% deposit rule, Firestore transaction and signed webhook. The source booking sequence that labelled an 80% deposit is intentionally not reproduced because it conflicts with the signed terms and the team's implemented pricing logic.

9.1.10.2 State Diagrams



Figure 10. Booking lifecycle states and permitted transitions.

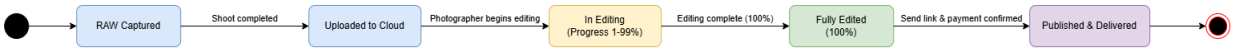
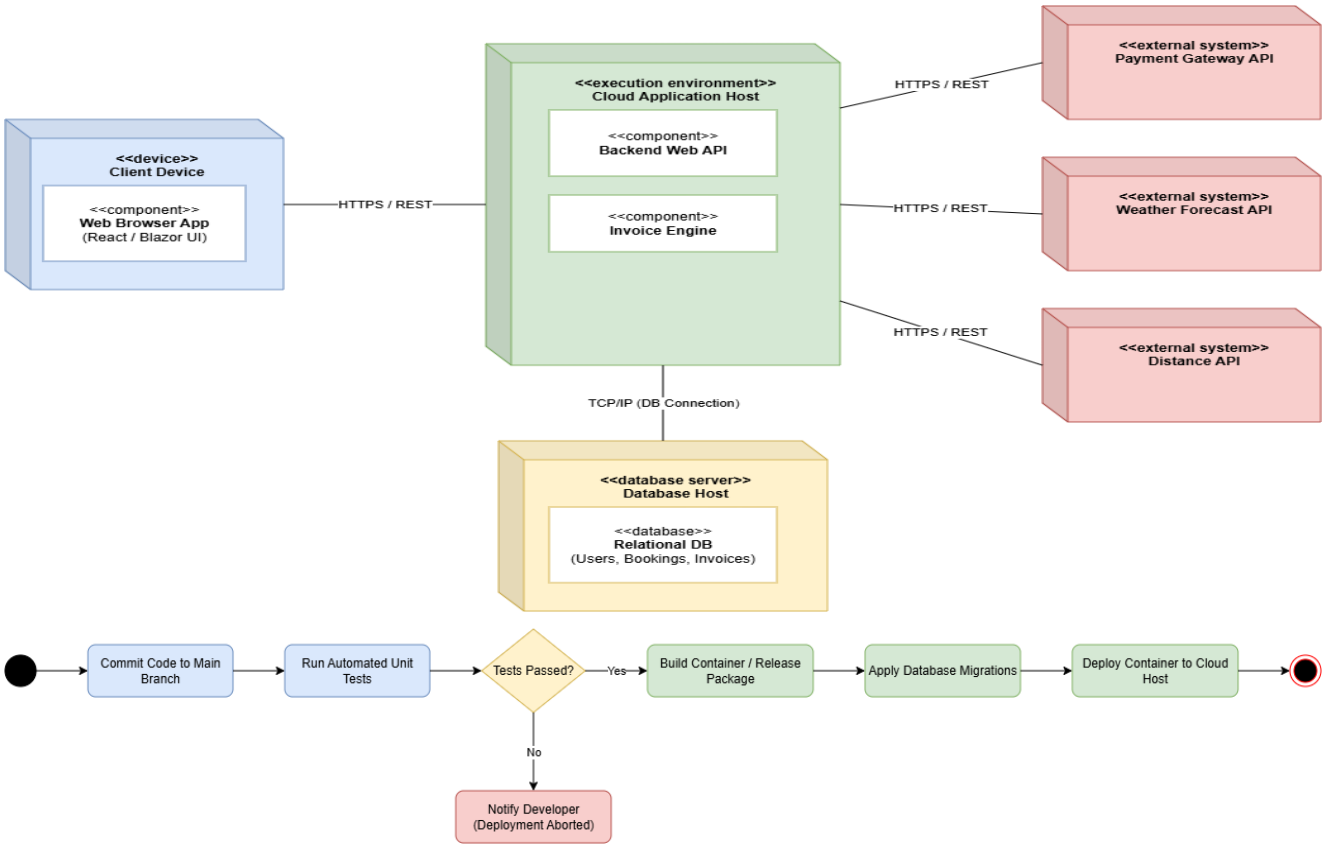


Figure 11. Photograph workflow from upload through editing and delivery.

9.1.10.3 Deployment Plan

Deploy a static Next.js export to Firebase Hosting and the containerised ASP.NET Core API to Cloud Run in africa-south1. Use separate staging and production Firebase/Google Cloud projects, least-privilege service identities and GitHub OIDC/Workload Identity Federation. The detailed service, trust-boundary and protocol deployment view appears in Figure 12; the gated promotion and rollback process appears in Figure 13.



9.1.11 Data Schema

9.1.11.1 Technology Choice and Conceptual ERD

Cloud Firestore is selected because it matches the team's Firebase decision, supports mobile/web SDK integration, managed scaling, transactions, indexes and regional deployment. Cloud Storage holds binary photographs; Firebase Authentication manages identities. A relational database would provide joins and stronger schema enforcement, but would introduce a separate platform and does not match the agreed implementation. The design compensates for NoSQL trade-offs through validation, document-version fields, deterministic slot locks and carefully bounded denormalisation (Firebase, n.d.-a; Firebase, n.d.-b).

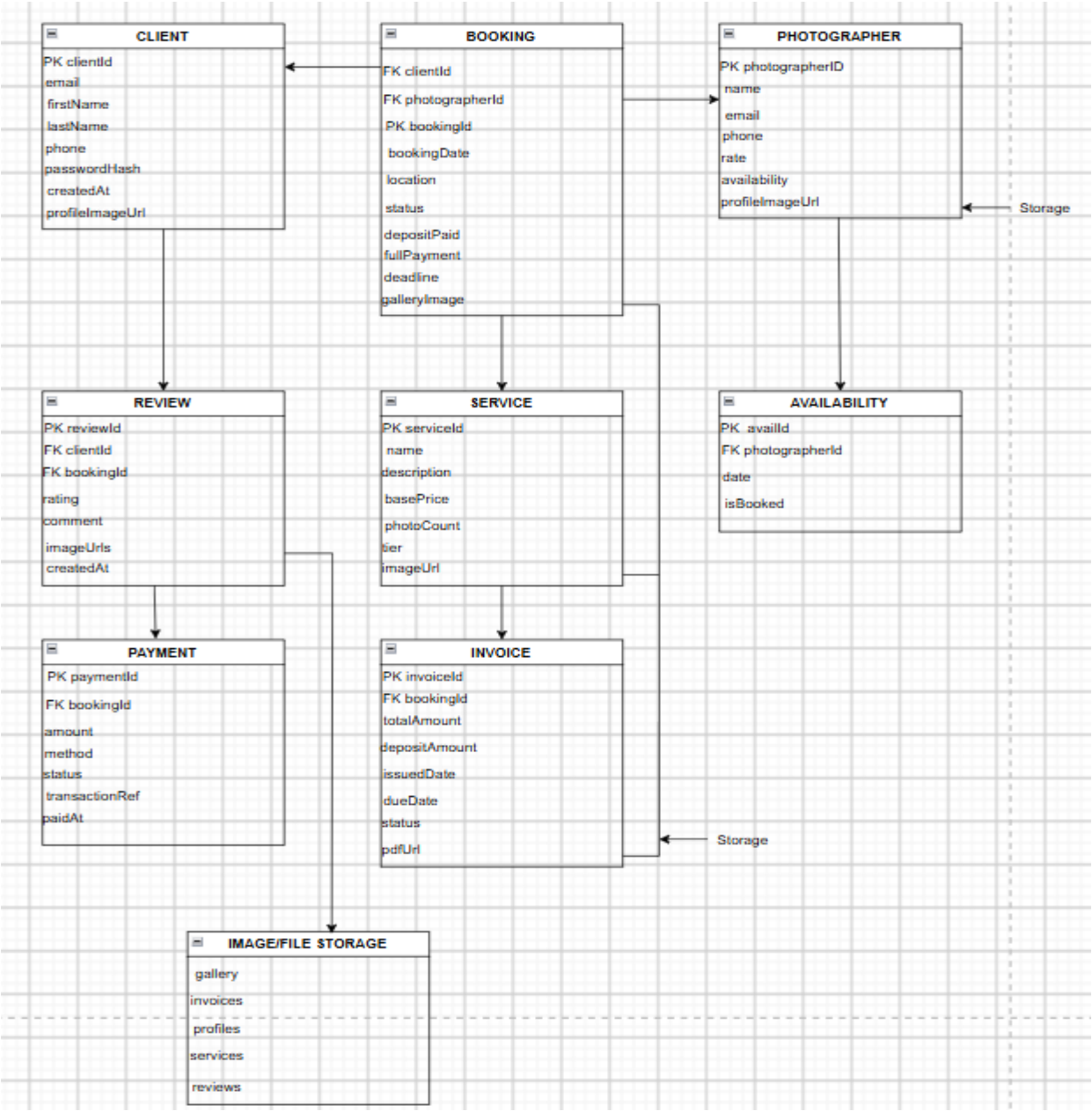


Figure 12. Conceptual entity-relationship model used to derive Firestore collections and references.

Table 4. Firestore collections and integrity rules.

Collection / path	Document key and core fields	Relationship / integrity rule
users/{uid}	uid, role, name, email, phone, status	UID comes from Firebase Authentication; role changes are administrator-only and audited.
bookings/{bookingId}	clientId, photographerId, serviceId, startUtc, endUtc, status, quoteSnapshot, termsVersion	References are validated server-side; status transitions follow Figure 6.
photographerSlots/{photographerId_startUtc}	bookingId, holdExpiresAt, state	Deterministic ID is atomically created/updated in a transaction to prevent conflicts.
payments/{providerEventId}	bookingId, amount, currency, provider, status, receivedAt	Provider event ID is the idempotency key; amount and booking are verified before confirmation.
mediaAssets/{assetId}	bookingId, ownerUid, objectPath, visibility, mimeType, checksum, retentionUntil	Object path is private by default; short-lived signed URLs and retention deletion apply.
outbox/{eventId}	aggregateId, eventType, payloadVersion, occurredAt, processedAt, attempts	Created with the booking/payment transaction; consumer retries are idempotent.

Fully Excusable Database schema

Page 49

9.1.11.2 Representative Firestore JSON Schemas

Booking document

```
{
  "schemaVersion": 1, "clientId": "uid-client",
  "photographerId": "uid-photo", "serviceId": "wedding-standard",
  "startUtc": "2026-11-14T08:00:00Z", "endUtc": "2026-11-14T14:00:00Z",
  "status": "CONFIRMED", "depositPercent": 50,
  "quoteSnapshot": {"currency": "ZAR", "total": 8500, "deposit": 4250},
  "termsVersion": "2026-08", "createdAt": "serverTimestamp"
}
```

Slot-lock document

```
{
  "schemaVersion": 1, "bookingId": "booking-123",
  "photographerId": "uid-photo", "slotStartUtc": "2026-11-14T08:00:00Z",
  "state": "CONFIRMED", "holdExpiresAt": null
}
```

Media asset document

```
{
  "schemaVersion": 1, "bookingId": "booking-123", "ownerUid": "uid-client",
  "objectPath": "private/bookings/booking-123/final/image-001.jpg",
  "visibility": "CLIENT_PRIVATE", "mimeType": "image/jpeg",
  "checksum": "sha256:...", "retentionUntil": "2027-02-14T00:00:00Z"
}
```

9.1.12 Architecture Artifacts

9.1.12.1 Design Patterns

The formal UML class-level that identifies interface , concreate classes , payments, operations and dependency/realisation relationships for the patterns applied to pricing, persistence, booking state and domain events.

Repository Pattern

Why it was selected: In order to separate the data access logic from the business logic, providing a clean separation between the application and the Firebase database. The selected design pattern is used since it provides a uniform interface for data access operations that does not depend on the particular data source.

How it is applied:

```
public interface IBookingRepository
{
    // Firestore operations
    Task<Booking> GetByIdAsync(string bookingId);
    Task<IEnumerable<Booking>> GetByClientIdAsync(string clientId);
    Task<IEnumerable<Booking>> GetByPhotographerIdAsync(string photographerId);
    Task<IEnumerable<Booking>> GetByDateRangeAsync(DateTime start, DateTime end);
    Task<bool> CheckAvailabilityAsync(string photographerId, DateTime date);
    Task<Booking> CreateAsync(Booking booking);
    Task UpdateAsync(Booking booking);
    Task DeleteAsync(string bookingId);

    // Cloud Storage operations
    Task<string> UploadGalleryImageAsync(string bookingId, Stream imageStream, string fileName);
    Task<IEnumerable<string>> GetGalleryImagesAsync(string bookingId);
    Task DeleteGalleryImageAsync(string imageUrl);
}
```

Dependency injection

Why it was selected: In order to decouple components and make them more testable (Martin, 2000). The selected design pattern is used since it promotes loose coupling through dependency injection. How it is applied:

```
// In Startup.cs/Program.cs
services.AddScoped<IBookingRepository, FirebaseBookingRepository>();
services.AddScoped<IPaymentService, YocoPaymentService>();
services.AddScoped<INotificationService, EmailNotificationService>();
services.AddScoped<IStorageService, FirebaseStorageService>(); // For Cloud Storage
```

Singleton Pattern

Why it was selected: In order to instantiate Firebase Firestore, Cloud Storage, and Yoco API clients only once and reuse them during the entire life cycle of the application. The selected design pattern is used since it ensure efficient resource management by managing connections to the database. How it is applied:

```
public static class FirebaseService
{
    private static FirestoreDb _firestoreDb;
    private static FirebaseStorage _firebaseStorage;
    private static readonly object _lock = new object();

    public static FirestoreDb FirestoreInstance
    {
        get
        {
            if (_firestoreDb == null)
            {
                lock (_lock)
                {
                    if (_firestoreDb == null)
                    {
                        _firestoreDb = FirestoreDb.Create("your-project-id");
                    }
                }
            }
            return _firestoreDb;
        }
    }
    public static FirebaseStorage StorageInstance
    {
        get
        {
            if (_firebaseStorage == null)
            {
                lock (_lock)
                {
                    if (_firebaseStorage == null)
                    {
                        _firebaseStorage = FirebaseStorage.Create("your-project-id");
                    }
```

```
            }
        }
        return _firebaseStorage;
    }
  }
}
```

Factory pattern

Why it was selected: In order to create different types of notification objects (email, WhatsApp, in-app) without concrete class instantiation. The selected design pattern is used since it encapsulates instantiation logic for notification channels. How it is applied:

```
public interface INotification
{
    Task SendAsync(string recipient, string subject, string message);
}

public class EmailNotification : INotification { /* Implementation */ }
public class WhatsAppNotification : INotification { /* Implementation */ }
public class InAppNotification : INotification { /* Implementation */ }

public class NotificationFactory
{
    public INotification CreateNotification(string channel)
    {
        switch (channel.ToLower())
        {
            case "email": return new EmailNotification();
            case "whatsapp": return new WhatsAppNotification();
            case "inapp": return new InAppNotification();
            default: throw new ArgumentException("Invalid notification channel");
        }
    }
}
```

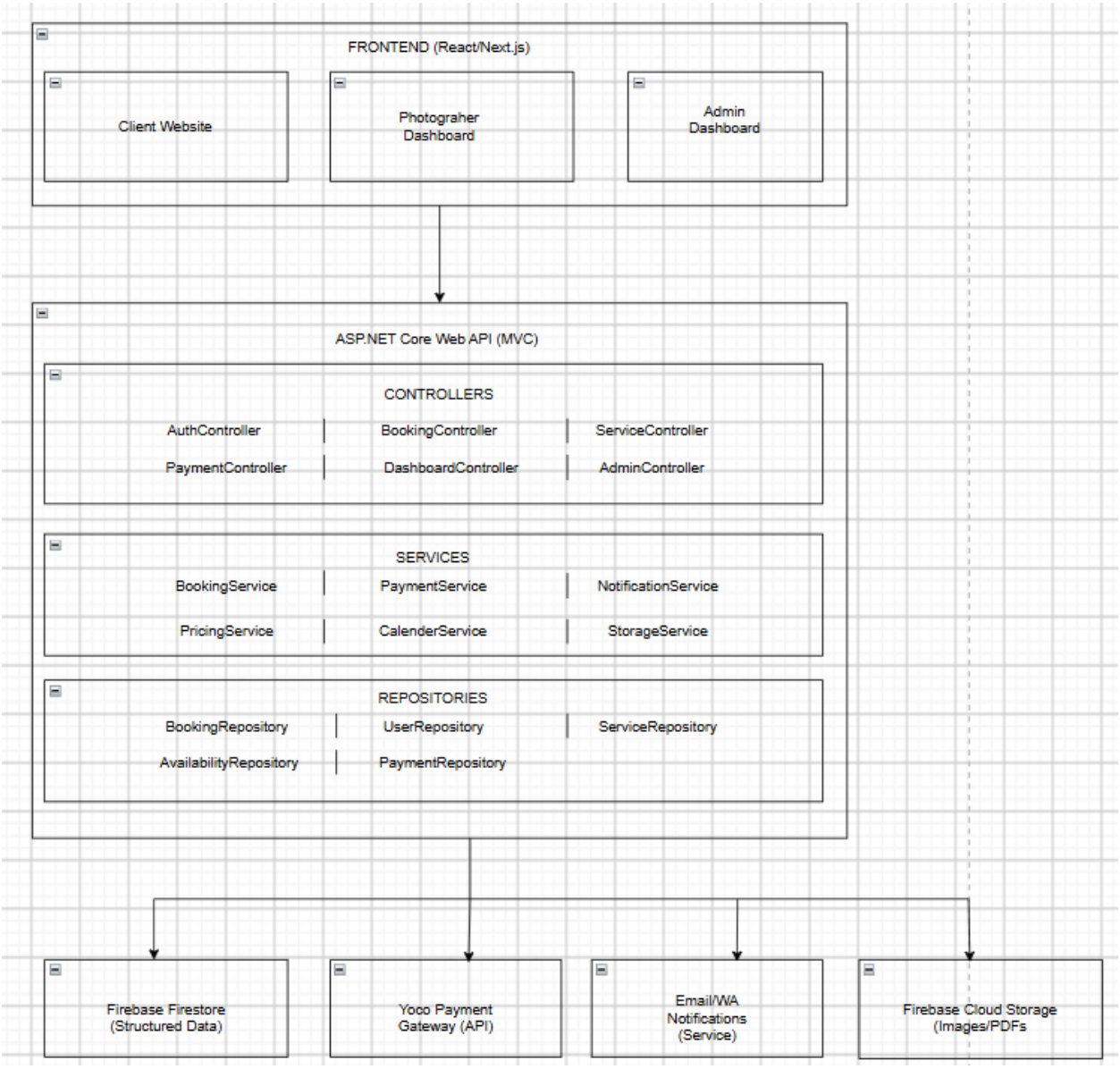


Figure 13. UML design-pattern realisation for the proposed system.

9.1.12.2 Architecture Patterns

9.1.12.2.1 Pattern selection and justification

A microservice architecture would add independent deployment, distributed transactions, service discovery and observability overhead before the small team has evidence that it needs them. A single unstructured application would be cheaper initially but would couple booking, payments and photographer workflow. The recommended middle ground packages one deployable .NET API while enforcing module ownership and inward dependency rules. This directly supports the documented scale and failure modes, reduces operational risk, and creates extraction seams if notifications, portfolio processing or reporting later require independent scaling (The Union Lab Wedding Photography & Camryn Burger Photography, n.d.; INSY7315 WIL Group, 2026a).

Table 5. Architecture pattern selection and trade-offs.

Pattern	Application in this system	Why it fit/trade-off controlled
Modular Monolith	Booking, Pricing, Payments, Workflow, Portfolio, Identity/Administration and Reporting are separate modules with explicit public contracts and owned Firestore collections/document prefixes.	Simple deployment and atomic consistency for bounded booking/payment transactions; boundary tests prevent a 'big ball of mud'.
Clean/ layered architecture	Presentation leads to Application use cases then that leads to Domain; infrastructure adapters implement interfaces defines inward.	Business rules such as deposit, cancellation and travel pricing can be unit-tested without Firebase or payment providers.
BBF/API Boundary	Next.js renders public pages and calls a purpose-built .NET API; browsers do not write authoritative Firestore booking/payment documents or hold privileged storage/provider credentials	Reduces exposed surface and keeps pricing, authorization and transactional slot claims server-side. Avoid putting business rules in React components.
Event-driven integration	A Firestore outbox document records BookingConfirmed, PaymentReceived and EditingDeadlineApproaching; an authenticated worker dispatches Cloud Tasks/Pub/Sub work.	Booking commits do not fail because WhatsApp/email is slow. At-least-once delivery requires idempotent consumers and a dead-letter queue.
CQRS-lite	Commands mutate aggregates; optimized queries feed availability, photographer dashboard and reports without separate databases initially.	Makes permissions and validation explicit without the complexity of full event sourcing.

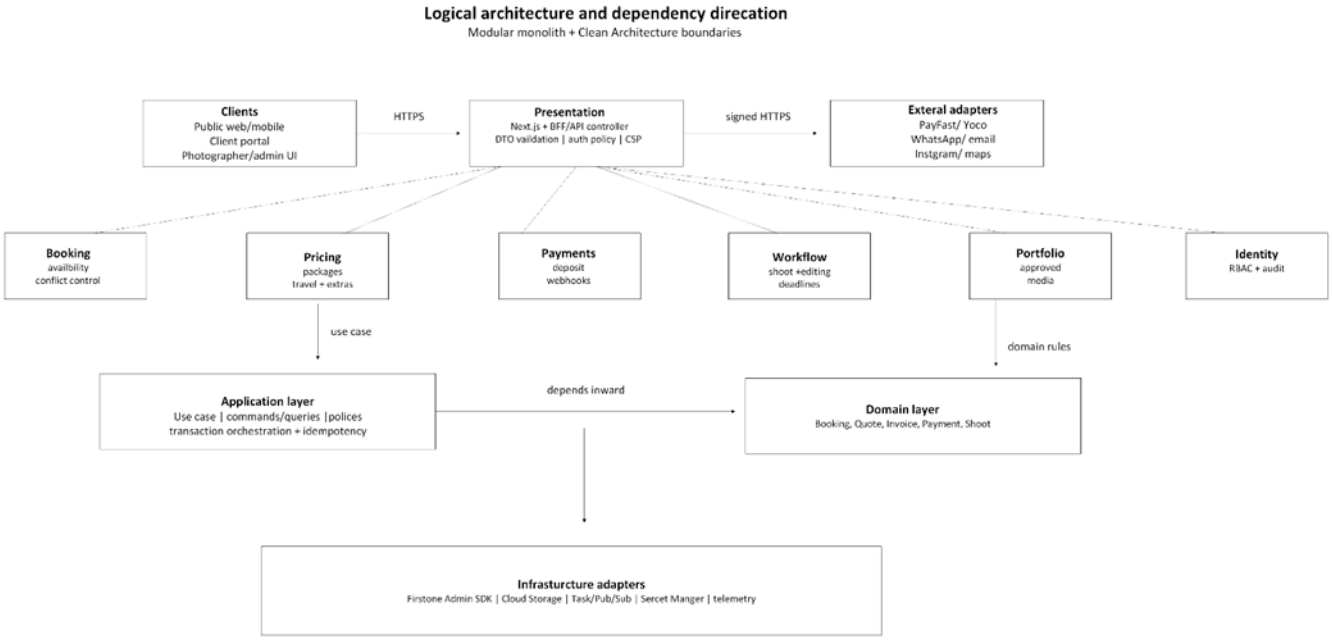


Figure 14. System architecture showing modules, layers, adapters and dependency direction.

9.1.12.2.2 Implemented website traceability

The supplied website is treated as a functional presentation/protype increment. Routes, controllers and business-rule calculations are implemented; authentication, persistence, payment confirmation, concurrency control and cloud integrations remain production-hardening work. This avoids claiming that mock behaviour already satisfies production security or reliability.

Table 6. Website prototype traceability to the production architecture.

Website evidence	Implemented behaviour	Architectural cross-reference	Production evolution required
Src/app/booking/page.tsx	Seven-step for service, package, photographer, date/time, location, client details, quote, terms acceptance and confirmation	Presentation Layer invoking Booking and Pricing use case; matches the critical flow in Figure 3.	Send commands to the .NET API; recalculate and the availability server-side; create a transactional hold before payment.
src/lib/api/booking.ts	Calculates R5/km beyond 20ks and a 50% deposit; creates/ cancels bookings and simulates Yoco/EFT payment	Prototype Strategy rules and Payment port; directly reflects the supplied terms	Move rules into versioned .NET strategies; replace localStorage and mock payment with Cloud Firestore plus a signed provider webhook.
src/app/client, photographer and admin/dashboard	Role-specific views for booking, invoices, rescheduling, shoot checklists, gallery links, revenue and outstanding balances	Presentation components for Client, Photographer and Administrator application queries/commands.	Enforces roles and resource ownership in API policies; expose only minimum DTOs; persist all charges and audit privileged actions.
Controllers/*.cs and Service/DemoDataStore.cs	ASP.NET endpoints for authentication, booking, services, photographers and dashboard data backed by singleton in-memory demo records	Current controller/infrastructure seam that can evolve into the Application and Repository/transaction structure in Design Pattern.	Add application services, Firestore repositories, deterministic slot-lock transactions, validation, idempotency and a Firestore outbox.
Src/lib/api/*.ts and src/lib/auth.ts	Frontend-facing API abstraction exists, but most functions use mock arrays/browser storage and browser-readable role/email cookies.	Useful adapter boundary: UI imports functions rather than embedding every data rule directly in components.	Replace implementations with typed HTTPS calls; use server-issued secure sessions/tokens and remove authority from localStorage/cookies readable by JavaScript.
Program.cs	ASP.NET serves APIs and, in development, starts/proxies the Next.js server while preserving /api routes.	Development composition/BFF seam between the Next.js presentation and .NET API.	Do not launch npm from the production API process; deploy the static Next.js export to Firebase Hosting and the containerised .NET API to Cloud Run as shown in Figure 2.
Src/app/terms/pages.tsx	Displays deposits, postponement/cancellation, travel and delivery policies and requires acceptance in the booking flow.	Policy presentation and evidence for pricing/state rules.	Version the accepted terms text and timestamp/hash per booking; reconcile wording with the signed client document before release.

This traceability demonstrates vertical synergy: the user-facing screens are not detached mock-ups; they correspond to the same Booking, Pricing, Payment, Workflow, Portfolio and Identity boundaries used throughout the architecture, security and DevOps plan (INSY7315 WIL Group, 2026b).

9.1.12.2.3 Logical component responsibilities

Table 7. Logical component responsibilities.

Component	Owns	Key rules and interactions
Booking	BookingHold, Booking, availability	Short-lived hold is created; database transaction prevents overlap; only confirmed after verified deposit.
Pricing	Package, PriceRule, Quote	It calculates base package, duration, extra photos at R5 each, travel beyond 20 km at R5/km, and approved adjustments. Version each rule used on a quote.
Payments	PaymentIntent, Payment, Invoice, Refund	Uses provider adapter; stores provider references/status only; verifies signed webhook; reconciles 50% deposit and final balance
Workflow	Shoot, EditingTask, Deadline, Assignment	Creates preparation/editing tasks, reminders and delivery deadline. Allows a photographer/admin to record approved reschedules.
Portfolio	PortfolioItem, MediaAsset	Publishes approved/consented work; imports social metadata through an adapter; never auto-publishes client media without approval
Identity/Admin	User, Role, StaffProfile, AuditEvent	Client/staff authentication; Customer, Photographer and Administrator roles; audit sensitive changes and exports.

9.1.12.2.4 Design-pattern integration

Table 8. Design-pattern integration.

Design pattern	Concrete use	Interaction / value
Strategy	IPriceRule implementations for package, travel, extra images, postponement and manual approved adjustment.	PricingEngine composes effective-dated rules; tests show the exact calculation and prevent inconsistent manual quotes.
Repository + transaction boundary	IBookingRepository and IPaymentRepository adapt the Firestore Admin SDK; one transaction owns each bounded booking command.	Domain logic is isolated; booking hold, claimed slots and outbox document commit atomically. Cross-command workflows use idempotent state transitions rather than pretending Firestore is relational.
Adapter / anti-corruption layer	IPaymentGateway for PayFast/Yoco; IMessageSender for WhatsApp/email; ISocialFeed for Instagram.	Provider payloads do not leak into the domain; provider can change without rewriting booking rules.
State	Booking: Draft -> Held -> AwaitingDeposit -> Confirmed -> ShootComplete -> Editing -> Delivered; cancellation/expiry paths are explicit.	Invalid transitions are rejected; permissions, deadlines and notifications are tied to a controlled lifecycle.
Observer via domain events	BookingConfirmed and PaymentReceived are persisted to an outbox and consumed asynchronously.	Decouples confirmation, invoice, reminder and reporting side effects; supports retry and audit.
Idempotency	Client idempotency key plus unique provider event ID.	Repeated clicks or webhook retries return the original result and cannot create duplicate charges/bookings.

9.1.12.2.5 Critical booking interaction

Combining availability checks, pricing calculations, and payment processing creates the application's highest-risk operational path. Front-end interfaces display optimistic availability, but final authority resides with the API and Firestore. Because Firestore lacks SQL-style exclusion constraints, the API normalizes booked timeframes into fixed 30-minute slot documents (photographerId/date/slot). A single Firestore transaction reads every required slot document, fails if any record reflects an existing hold or confirmation, and atomically creates the hold with all associated slot claims. When concurrent requests collide, Firestore re-runs the transaction, causing rival operations to detect the claims and reject. Finally, an authenticated payment webhook idempotently updates the hold to Confirmed and writes the outbox event (Firebase, n.d.-b).

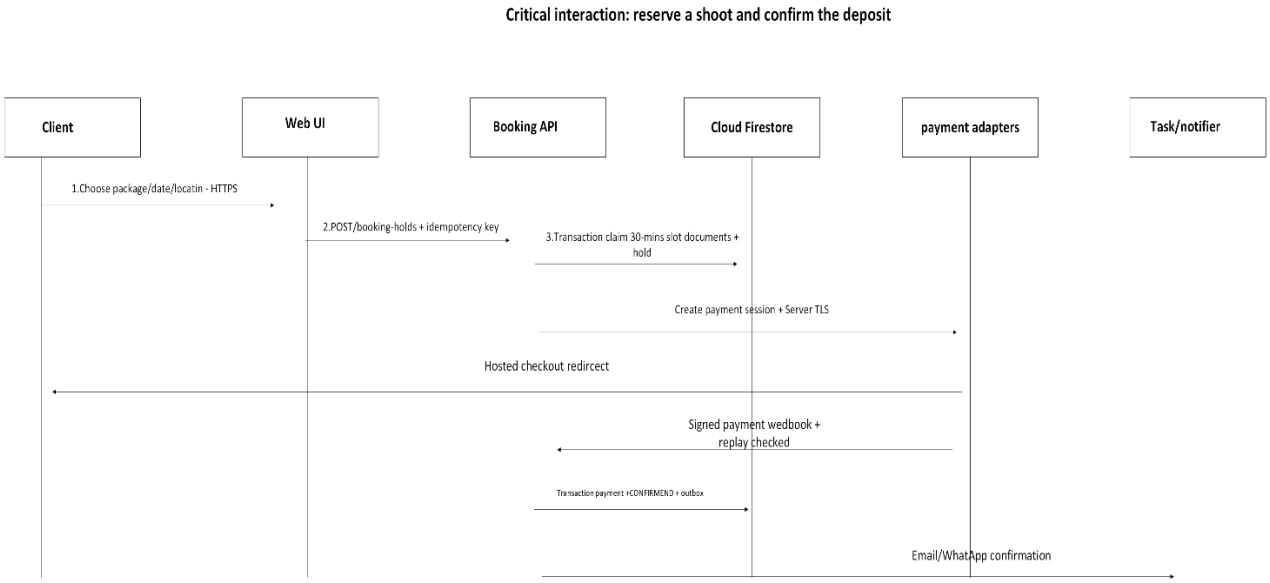


Figure 15. Booking hold, hosted deposit and idempotent confirmation sequence.

9.1.12.2.6 Evolution and architecture acceptance tests

- Module tests fail if one module reads another module's tables directly; cross-module access is through application contracts or events.
- Concurrency test launches competing reservations for the same photographer/time; exactly one may confirm.
- Contract tests replay PayFast/Yoco and WhatsApp/email sandbox payloads, including invalid signatures and duplicates.
- Extract a module only after telemetry shows an independent scaling/release need; Notifications and Portfolio are the first candidates.
- Architecture decision records (ADRs) capture stack, pattern, provider and network decisions with alternatives and consequences.

9.1.12.3 Cloud Architecture

The recommended baseline uses separate Firebase/Google Clous projects for staging and production. Deloy the static Next.js export to Firebase Hosting, the ASP .NET API to cloud Run, identity to Firebase Authentication, operational documents to cloud Firestore and image to Cloud Storage for Firebase. Add App Check, Secret Manager, Cloud Task/Pub/Sub, Cloud Logging/Monitoring and infrastructure/rules as code. Firebase Hosting can route HTTPS paths to Cloud Run supports the existing containerised .NET API; these preservers the demonstrated Next.js/.NET separation without operating Kubernetes (Firebase, n.d.-d; Google Cloud, n.d.-a).

9.1.12.3.1 Service selection and justification

Table 9. Cloud service selection and security justification.

Service	Role and configuration	Justification/security rationale
Firebase Hosting	Custom domain, managed TLS and global CDN for the static Next.js export; security headers; /api/** rewrite pinned to a Cloud Run revision.	Low operational overhead and fast portfolio delivery. A pinned rewrite keeps Hosting content/config and the API revision aligned and supports coherent rollback (Firebase, n.d.-d).
Cloud - .NET API	Containerised C# API, /health endpoint, min/max instances, concurrency cap, runtime service account, Secret Manager references and structured logs	Runs the existing .NET stack without cluster administration and scales to demand. It is the trusted boundary for pricing, authorization, slot transactions and provider calls.
Firebase Authentication +App check	Email/password or approved provider; MFA for privileged staff; custom claims for coarse roles; API verifies ID token, revocation where sensitive, and App Check token.	Authentication identifies the user; API resource checks provide authorization. App Check reduces abuse from clients that are not the registered app but does not replace authentication (Firebase, n.d.-e; Firebase, n.d.-f).
Cloud Firestore (NoSQL)	Collections for users, services, priceRuleVersions, bookings, photographerSlots, payments, tasks, outbox and auditEvents; composite indexes; PITR/backups; runtime service account via IAM	Real-time document reads and atomic transactions suit the selected Firebase platform. Deterministic slot documents replace the SQL uniqueness/exclusion constraint and prevent overlapping reservations under contention (Firebase, n.d.-a; Firebase, n.d.-b).
Cloud Storage for firebase	Private portfolio/galleries/invoices; random object names; upload size/type rules; quarantine/scanning flow; lifecycle, versioning and retention; short-lived signed URLs.	Object storage fits large media. Security Rules can authorize paths and validate contentType/size, while signed URLs avoid public buckets and API media proxying (Firebase, n.d.-g).
Cloud task/Pub/Sub + worker	Outbox dispatcher creates authenticated tasks/events; bounded retry, idempotent handlers and dead-letter handling for reminders, invoices and notifications.	Provider failure does not roll back a booking, and independently scalable consumers prevent long work from exceeding Firebase Hosting's request timeout.
Secret Manger + service accounts	Payment, messaging and social-provider secrets; dedicated runtime/deployment identities; minimum IAM; rotation and audit.	Secrets stay outside source and Firebase web configuration. Cloud Run's attached service identity avoids distributing a service-account key.
Cloud Logging/Monitoring+ Audit logs	Distributed traces, metrics, structured application/security logs, alerts and dashboards; correlation IDs and PII redaction.	Provides operational, incident and release evidence without logging tokens, image URLs or payment payloads.

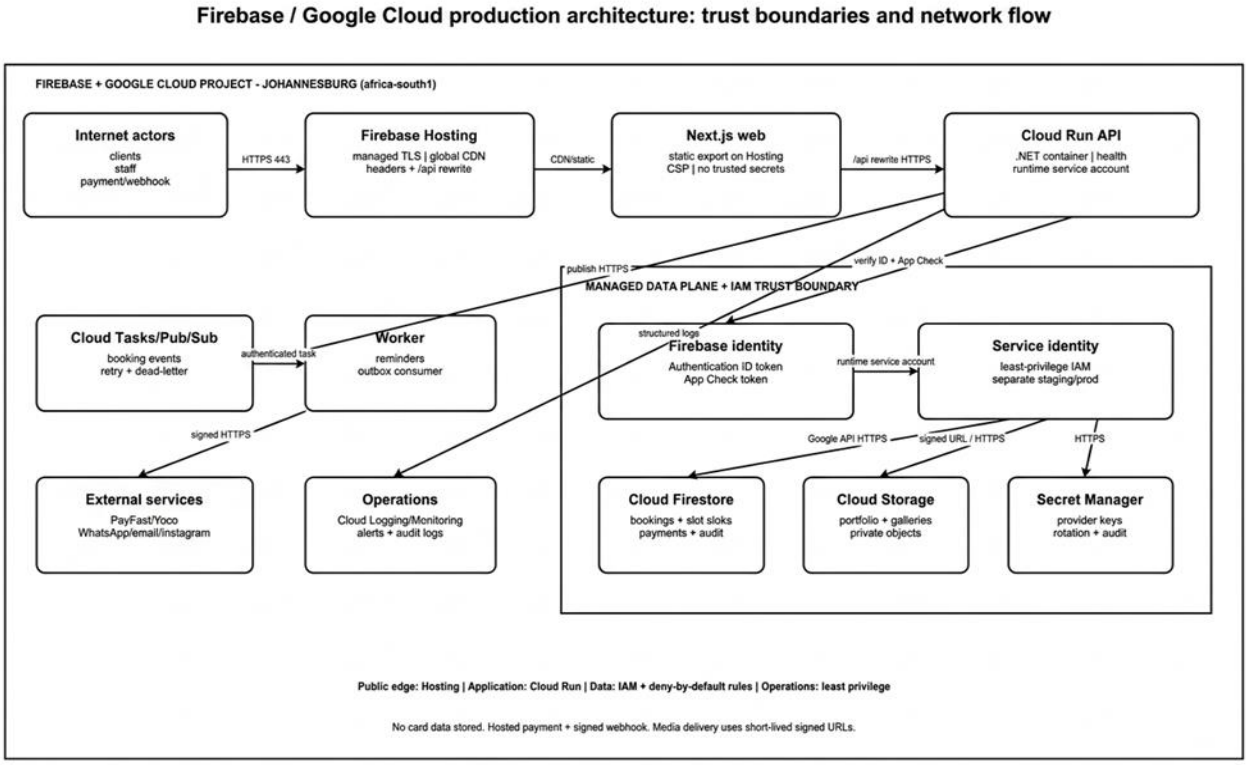


Figure 16. Firebase and Google Cloud trust boundaries, services and network flow.

9.1.12.3.2 Network and protocol flow

1. Client and staff browsers resolve the custom domain to Firebase Hosting and use HTTPS on TCP 443. Hosting terminates TLS, serves the static Next.js export from its CDN and returns HSTS, CSP, anti-framing and MIME-sniffing headers.
2. Static assets are served directly; /api/** is rewritten over managed HTTPS to the Cloud Run API. The browser sends a Firebase ID token in Authorization and an App Check token. The API verifies both, then enforces role plus resource ownership before applying domain rules.
3. Authoritative browser writes to booking, payment and audit collections are denied. The .NET API uses Application Default Credentials from its dedicated Cloud Run service account. Firestore server libraries bypass Firebase Security Rules, so least-privilege IAM on that runtime identity is mandatory (Firebase, n.d.-h).
4. The API reaches Firestore, Cloud Storage and Secret Manager through encrypted Google APIs. This serverless baseline does not falsely claim private-service endpoints that are absent from the design: separation is achieved with distinct staging/production projects, isolated runtime/deployment service accounts, IAM, deny-by-default client rules and explicit egress/provider allow-lists.
5. For a booking, the API atomically writes the hold, all deterministic photographer-slot claims and an outbox document. A dispatcher submits authenticated Cloud Tasks or Pub/Sub messages; workers use bounded retry, idempotency and dead-letter handling.
6. Payment is provider-hosted. The API creates a session using server-to-server HTTPS; the provider calls a dedicated webhook over HTTPS. The application validates signature, expected amount/currency, timestamp and event uniqueness before changing state.
7. WhatsApp/email/social integrations use outbound HTTPS 443 from an allow-controlled path. No third party receives more client data than the message or transaction requires.
8. The API creates short-lived signed media URLs only after checking gallery ownership/consent; direct client uploads, if later enabled, require Firebase Authentication, App Check and rules that restrict path, content type and size. New uploads enter quarantine until scanning and EXIF removal complete.

9. Resources send redacted telemetry to Cloud Logging/Monitoring. Alerts target elevated 5xx/latency, token failures, abnormal Firestore/Storage denials or cost, task age/dead letters, payment reconciliation and backup failures.

9.1.12.3.3 Availability, scaling and recovery

Table 10. Availability, scaling and recovery controls.

Concern	Design	Verification / trigger
Demand spikes	Hosting CDN caches public assets; Cloud Run scales within cost-safe min/max/concurrency limits; tasks absorb notification bursts.	Load-test at 3x expected peak; verify exactly-one booking under Firestore contention and monitor document hot spots.
Transient failure	Timeouts, exponential backoff with jitter and circuit breakers at external adapters; Tasks/Pub/Sub retry and dead-letter.	Chaos/sandbox tests for payment and messaging outage; booking remains queryable.
Database recovery	Enable Firestore PITR and scheduled backups with protected restore roles and a documented clone/restore/reconciliation runbook.	Quarterly restore drill proves the RPO <=15 min/RTO <=4 h target or records a gap; Firebase documents a seven-day PITR window when enabled (Firebase, n.d.-i; Firebase, n.d.-j).
Media recovery	Cloud Storage object versioning/retention/lifecycle; three-month client image rule implemented by a reviewed retention job with legal/business hold override.	Monthly retention report; sample restore; deletion audit.
Release failure	Deploy immutable Cloud Run revision with no traffic, run health/smoke, then cut traffic; Hosting pinTag aligns rewrite and API revision.	Rollback traffic or the Hosting release to the previous pinned revision on health/error threshold.

9.1.12.3.4 Assumptions and decisions to confirm

- It is confirmed that Camryn Burger Photography, The Union Lab, or a dual-brand domain
- PayFast and Yoco will be used

9.1.13 Security

Apply defense in depth; managed TLS/CDN, secure-by-default application code, Firebase Authentication plus App Check, API resources authorization, deny-by-default client rule, least-privilege runtime IAM, protected CI/CD, tamper-evident auditing, tested recovery and incident response. App Check and rate limiting are compensating controls, not substitutes for output encoding and validation. The control set is organized so that it can be verified rather than merely asserted: OWASP ASVS provides a structured basis for testing application and environmental security controls, including controls relied on against XSS and injection (OWASP, n.d.-b).

9.1.13.1 Assets, trust boundaries and principles

Sensitive assets include client contact details, booking locations/dates, contracts, invoices, payment status/provider references, staff schedules, image links and credentials. Card details must remain with the payment provider. Trust boundaries occur at Firebase Hosting, browser/API, Cloud Run/Firebase managed data, asynchronous workers, staff administration, third-party webhooks and the GitHub-to-Google Cloud deployment boundary.

- **Zero trust between components.** Authenticate every call, use dedicated service accounts for Google Cloud resources, verify Firebase ID/App Check tokens, validate third-party messages and scope IAM/egress.
- **Least privilege and deny by default.** Separate Customer, Photographer and Administrator roles; resource-level checks ensure a client sees only their bookings and a photographer sees assigned work. Privileged actions require MFA and fresh authentication.
- **Secure failure.** Errors reveal no stack traces or account existence; failed notifications do not undo a booking; payment inconsistency is quarantined for reconciliation.
- **Data minimization.** Collect only fields needed for booking/delivery, avoid sensitive data in logs, define retention/deletion, and obtain explicit portfolio/marketing consent.
- **Accountability.** Audit login, role, pricing override, reschedule, cancellation, payment/refund, data export and deletion events with actor, time, object and correlation ID.

9.1.13.2 Prototype security delta

Rigorous security evaluation requires drawing a clear line between client-side visual cues and true, backend-enforced authorization rules. Although the prototype successfully isolates distinct user roles within the presentation layer, its session handling and data persistence mechanisms are purely illustrative by design. Consequently, the source-referenced gaps must be integrated into the product backlog as defined acceptance criteria rather than categorized as undisclosed vulnerabilities (INSY7315 WIL Group, 2026b).

Table 11. Prototype security gaps and required production controls.

Observed prototype condition	Risk created	Required production control / acceptance criterion
src/lib/auth.ts stores role/email in localStorage and JavaScript-readable cookies; role redirects occur in useEffect.	A user can alter browser values, assume another role or expose session data through XSS.	Use an established server-side identity provider/session; Secure, HttpOnly, SameSite cookies; MFA for staff; server policies on every protected endpoint. Client redirects are usability only.
AuthController accepts demo/fallback users, does not verify the submitted password and returns demo token strings.	Authentication bypass and privilege selection if exposed beyond demonstration use.	Remove demo fallback in non-development builds; hashed passwords or external OIDC; generic failure messages; rate limiting; token rotation/revocation; automated negative tests.
Controllers expose booking read/create/update/delete and dashboard data without [Authorize] or ownership policies; UseAuthorization is configured without authentication registration.	IDOR, bulk data disclosure and unauthorized modification/deletion.	Add authentication middleware before authorization; policy/role attributes; resource-level ownership/assignment checks; separate admin commands; audit and integration tests for cross-user denial.
Bookings, profiles and gallery links use singleton memory, module arrays or localStorage.	Data disappears on restart, differs between browsers and can be modified outside business rules.	Firestore/Cloud Storage persistence; repository and transaction boundary; slot claims; validation; backup/retention; browser stores only non-authoritative UI state.
createBooking marks the booking Confirmed and Deposit Paid; createPayment always succeeds; availability returns predetermined slots.	False payment confirmation and double booking; pricing/status can be manipulated client-side.	Server recomputes quote, creates a time-limited hold, accepts only verified signed payment webhooks, reconciles amount/currency and confirms in one transaction.
saveGalleryLink accepts an arbitrary external URL and portfolio images load from remote hosts.	Phishing/open redirect, unsafe schemes, privacy leakage or unapproved media publication.	HTTPS allow-list or private Cloud Storage galleries, URL canonicalization, consent record, short-lived signed URL, CSP image/connect allow-lists and operator review.
appsettings AllowedHosts is '*' and Program.cs forwards most request headers to the development Next.js proxy.	Host-header/proxy ambiguity and accidental development behavior in production.	Environment-specific allowed hosts; trusted proxy/forwarded-header configuration; strip hop-by-hop/security-sensitive headers; production build must disable process launch and dev proxy.

9.1.13.3 Comprehensive security risk register

Table 12. Security risk register and mitigation strategies.

ID / risk	Likelihood / impact	Mitigation strategy	Owner	Verification / residual monitoring
S1. Stored/reflected XSS in names, notes, portfolio captions or imported social content	H/H	React auto-escaping; context-aware output encoding; sanitize allow-listed rich text; CSP with nonces and no unsafe-inline; HttpOnly/Secure/SameSite cookies; API validation; XSS unit/DAST tests.	App lead	No executable markup in persistence/render tests; CSP violation alerts reviewed.
S2. NoSQL query manipulation, mass assignment or unsafe document paths through booking/search/admin APIs	H/H	Never accept collection/path names from clients; DTO allow-lists and type/range validation; parameterized SDK queries; query limit/index policy; least-privilege runtime IAM; SAST/DAST.	API lead	Negative query/path suite passes; runtime identity cannot administer projects, rules or indexes.
S3. Broken access control / IDOR exposes another client's booking, invoice or images; current controllers have no enforceable authorization	H/H	Verify Firebase ID token; central policy authorization; ownership/assignment checks on every object; deny authoritative client Firestore writes; short-lived signed media URLs; never trust browser role/custom-claim alone for resource ownership.	Security owner	Cross-user integration tests return 403/404; emulator rules tests deny cross-user access; protected-route inventory complete.
S4. Prototype login ignores passwords and stores role/email in browser-readable storage; credential/session attack in production	H/H	Replace demo login; MFA for staff/admin; modern identity provider; rate limits and lockout/risk detection; Secure/HttpOnly/SameSite cookies, rotation and short idle timeout; re-auth for exports/refunds.	Identity owner	Demo paths absent from production build; MFA coverage 100% privileged users; anomalous login alerts.
S5. CSRF changes booking, reschedule or profile through authenticated browser	H/M	SameSite cookies plus anti-forgery token for state-changing cookie-auth requests; validate Origin/Referrer; never mutate on GET.	Web lead	Automated CSRF negative tests on all mutation routes.
S6. Mock payment currently always succeeds and marks Deposit Paid; forged/replayed webhook or price manipulation could confirm unpaid booking	H/H	Hosted checkout; server computes amount; verify provider signature and timestamp; IP rules if supported; unique event ID/idempotency; compare currency/amount/reference; reconciliation job.	Payments owner	Mock provider blocked outside development; invalid/replayed events rejected and alerted; daily settlement exceptions zero.
S7. Predetermined availability and client-side confirmation can create a double booking under concurrency	H/H	Database-enforced conflict rule/transaction isolation; short hold with expiry; idempotency key; concurrency tests; never trust cached UI availability.	Booking owner	Parallel test confirms exactly one booking;

ID / risk	Likelihood / impact	Mitigation strategy	Owner	Verification / residual monitoring
				conflict metric monitored.
S8. Malicious file upload or media leaks GPS/identity metadata	H/M	Allow-list MIME/extension; verify magic bytes; size/dimension limit; malware scan; random object name; private container; strip EXIF; explicit publication consent.	Portfolio owner	Upload fuzz tests; scan failures quarantined; consent recorded.
S9. Secrets or service-account keys leak from repository, logs or CI	H/M	Secret Manager; attached runtime identity; GitHub OIDC with Workload Identity Federation; secret scanning/push protection; masked logs; minimum workflow permissions; rotate/revoke on alert.	DevOps owner	No service-account JSON key or long-lived cloud secret; quarterly IAM/rotation review.
S10. Dependency/supply-chain compromise	H/M	Lock files; Dependabot; dependency review; CodeQL/SAST; SBOM; pin third-party actions to commit SHA; verify provenance; block high/critical new vulnerabilities.	DevOps owner	PR gate and scheduled scan; documented exception expiry.
S11. DoS, bots or scraper abuse availability and Firebase/Cloud Run costs	M/H	Hosting caching; App Check; per-user/IP/route API rate limits; request/body limits; Cloud Run max instances; task back-pressure; budget/quota alerts; reCAPTCHA Enterprise step-up on abuse signals.	Cloud owner	Load test and cost/429/task-age/App Check failure alerts.
S12. Personal data or role state stored in localStorage/browser cookies, or images disclosed through rules, logs, backups, signed links or misconfiguration	H/M	Remove authoritative browser storage; deny-by-default Firestore/Storage rules; runtime IAM; redaction; short-lived signed URLs; IAM reviews; backup access control; rules/IaC tests.	Data owner	Browser inspection contains no booking/session authority; emulator tests prove default deny; quarterly restore/access audit.
S13. Insider misuse: pricing override, export, cancellation or refund	H/M	Segregation of duties for high-value refunds/exports; reason codes; immutable audit; just-in-time privileged access; alerts and periodic review.	Business owner	Monthly privileged-action report and sampled approval evidence.
S14. Singleton/local mock data loss or reminder/integration outage causes lost booking, missed shoot or deadline	H/M	Replace DemoDataStore/local arrays with durable Firestore; transactional outbox + Tasks/Pub/Sub retry/dead-letter; provider circuit breaker; dashboard remains source of truth; operator alert/manual resend runbook.	Operations	Restart/persistence test; task age/dead-letter alerts; recovery drill and delivery-rate SLO.
S15. Data loss/ransomware or incorrect retention deletion	H/M	Firestore PITR and scheduled backups; Storage object versioning/retention; protected restore roles; tested restore; retention-job dry run and approval for bulk deletion; recovery identity separated.	Cloud owner	Quarterly restore; deletion report; backup alert.

ID / risk	Likelihood / impact	Mitigation strategy	Owner	Verification / residual monitoring
S16. Deployment pipeline compromise or unauthorized production release	H/M	Protected main; required checks/reviews; CODEOWNERS; OIDC; environment approval; SHA-pinned actions; least permissions; artifact promotion; signed audit trail.	DevOps owner	Only protected main artifact deploys; review and SHA recorded.

9.1.13.4 XSS prevention in depth

According to OWASP (n.d.-a), robust defense against Cross-Site Scripting (XSS) necessitates a multi-layered security posture comprising framework-native rendering, context-specific output encoding, HTML sanitization, and safe sink handling. Content Security Policy (CSP) serves strictly as a defense-in-depth mechanism and must not be relied upon as a primary substitute for contextual output encoding

10. Treat all client, staff and social-feed values as untrusted. Store canonical text; reject unexpected markup at the API boundary. If rich text is genuinely required, sanitize server-side with a maintained allow-list and never allow script, style, event-handler, iframe or dangerous URL schemes.
11. Render with React/Next.js text binding, which escapes by default. Prohibit dangerouslySetInnerHTML except in one reviewed sanitizer component. Use context-specific encoding for HTML, attributes, URLs and JavaScript; URL allow-list external links.
12. Send a restrictive Content-Security-Policy: default-src 'self'; script-src 'self' with per-response nonces; object-src 'none'; base-uri 'self'; frame-ancestors 'none'; form-action 'self' plus explicitly required payment destinations; img-src 'self' data: and approved storage/social hosts. Avoid unsafe-inline/unsafe-eval.
13. Use HttpOnly, Secure, SameSite cookies; HSTS; X-Content-Type-Options: no sniff; Referrer-Policy; Permissions-Policy. CSP and App Check reports are monitored, but application encoding remains the primary control.
14. Test payloads in names, notes, portfolio captions, query strings, error messages and imported social text. CI runs unit tests for sanitizer/encoding and DAST against staging; a failure blocks promotion.

9.1.13.5 Data protection, privacy and retention

Table 13. Data protection and retention decisions.

Data class	Examples	Protection and retention decision
Restricted	Authentication secrets, provider keys	Secret Manager only; never logged; service-account access; rotate on exposure and at provider policy interval.
Confidential	Client contact/location, booking, invoice, contracts, staff schedules	TLS; encrypted managed storage; least-privilege row/resource checks; audit exports; retain only for agreed business/legal period.
Sensitive media	Private galleries, image links, raw/edited photos	Private Cloud Storage; short-lived signed URLs; consent and access logging; align deletion with the stated three-month image-storage term, subject to confirmed legal/accounting holds.
Public	Approved portfolio, services, terms, about	Publish only reviewed and consented content; integrity via controlled admin workflow and CDN cache purge.

To meet POPIA requirements, South African data privacy controls should be validated with the client or legal advisors. The technical architecture needs to implement mechanisms for processing subject rights requests (access, correction, deletion), managing security compromises, mandating operator contracts, and reviewing foreign data transfers. This document provides technical implementation guidance only and does not constitute legal counsel.

9.1.13.6 Security operations and incident response

15. Detect: correlate Firebase Authentication/App Check, API, Firestore/Storage audit, task, payment and pipeline events using a shared correlation ID; alert only on actionable thresholds.
16. Triage: confirm severity, affected assets/users and whether bookings/payments are incorrect. Preserve logs and deployment evidence; assign incident commander and communications owner.
17. Contain: disable compromised account/token, revoke signed-link access or rotate its signing identity, rotate provider secret, block/rate-limit abusive traffic, pause risky deployment or isolate integration without deleting evidence.
18. Eradicate and recover: patch, validate in staging, restore/reconcile data if required, deploy through the controlled pipeline, and monitor an enhanced observation window.
19. Learn: document root cause and timeline, notify affected parties/regulators where required, add regression controls/tests, and track remediation to closure.

9.1.14 DevOps

The operating uses trunk-based GitHub flow with short-lived feature braces, build-once artifact promotion, pull-request quality gates, and approved production Cloud Run revision cutover/Firebase hosting revision and automated to a separate staging Firebase project. Application code, Firebase rules/ indexes and infrastructure follow the same review/audit model.

9.1.14.1 Version control, branching and collaboration

Table 14. Version-control rules and supporting evidence.

Element	Rule	Reason / evidence
main	Always releasable; direct pushes and force-pushes blocked; required PR, checks and CODEOWNERS review.	Creates a single auditable source of production truth.
feature/<issue>-<slug>	Short-lived branch from current main; small commits; rebase/update before merge; squash merge.	Limits merge drift and makes rollback/review easier.
release tags	Immutable semantic tag points to the already-tested main commit/artifact; no separate long-lived release branch.	Prevents environment-specific rebuilds and branch divergence.
hotfix	Branch from production tag, run the full pipeline, PR to main, then redeploy the resulting artifact; record incident link.	Tests or audit, Urgency does not bypass review,.
Definition of done	Acceptance criteria met; tests and threat-sensitive cases updated; migration is compatible; telemetry/runbook/documentation updated.	Operations and security are part of delivery, Quality, not post-release work

9.1.14.2 GitHub Actions Pipeline

Table 15. Repository evidence and CI/CD alignment.

Repository evidence	**Current state**	**Pipeline/document alignment**
CB Photography Webapp.csproj	ASP.NET Core web project targets net10.0 and references SpaProxy.	Appendix A now uses .NET 10 and the real project filename rather than a hypothetical src/api solution.
package.json and package-lock.json	Next.js 16.3.0, React 19.2.8, ESLint and TypeScript are rooted beside the .NET project; build/lint scripts exist.	Workflow runs npm ci, ESLint, tsc and next build from repository root; the committed lock file supports deterministic npm restoration.
Program.cs	Development-only middleware starts/proxies Next.js /api is handled by controllers.	CI builds both stacks; production deploys pre-built web/API artifacts and must not run the development child-process launcher.
No test project/test script or workflow in supplied archive	Functional pages and demo APIs exist, but automated test evidence is not yet present.	Pipeline executes available checks immediately and the definition of done requires pricing, authorization, booking-concurrency and provider-contract test projects before production approval.
No firebase.json, Firestore/Storage rules, Dockerfile or Google Cloud IaC in supplied archive	Firebase cloud architecture is still a Task 1 deployment plan.	Rules/indexes/IaC, Cloud Run container, environment protection and Google Workload Identity Federation remain explicit backlog items, not falsely claimed as completed.

This alignment makes the DevOps plan auditable against the team artefacts: runtime, each workflow path and identified missing quality gate deriver from the submitted archive rather than generic sample (INSY7315 WIL

Group, 2026b). The GitHub environments can restrict deployment branches, withhold environment secrets until protection rule pass and require reviewers; concurrency prevents overlapping deployment. . OIDC requires id-token: write and exchanges a workflow identity for short-lived cloud credentials instead of a stored cloud secret (GitHub, n.d.-a; GitHub, n.d.-b; GitHub, n.d.-c).

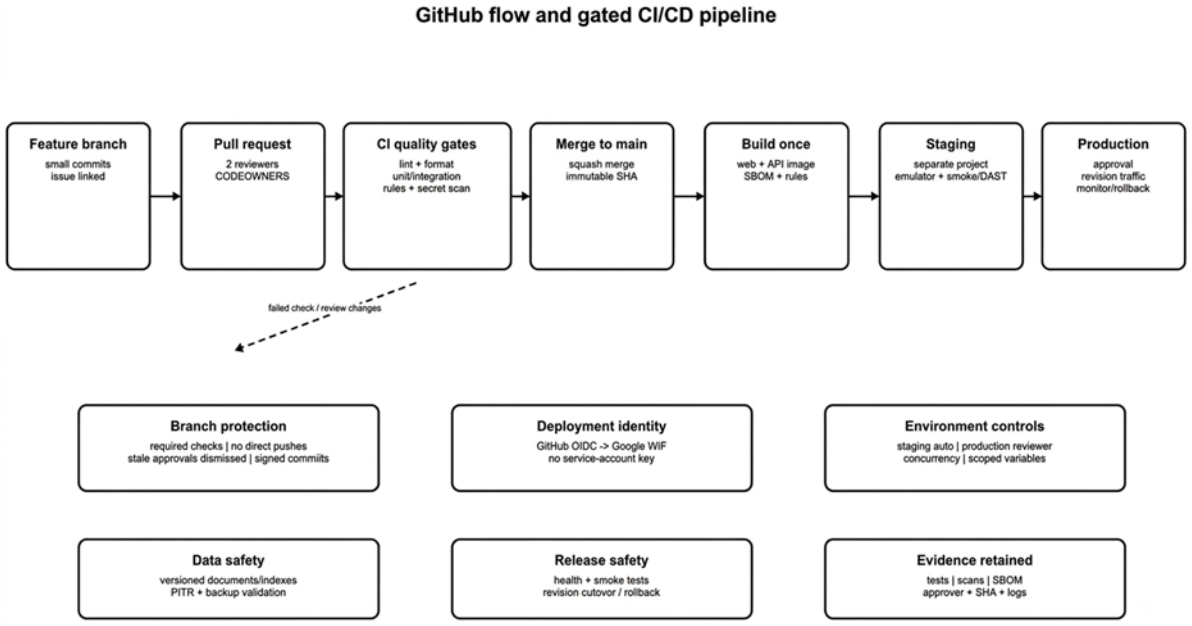


Figure 17. GitHub branching and gated CI/CD workflow.

Table 16. GitHub Actions stages, gates and outputs.

Stage / trigger	Automated work	Gate and output
PR validation	Checkout read-only; restore locked dependencies; lint/format; Next.js build/type checks; .NET build; unit tests plus Firebase Emulator Suite tests for Firestore/Storage rules and booking contention; coverage report.	All required checks pass; rules are deny-by-default; exactly one competing booking succeeds; artifacts are not deployed.
PR security	CodeQL/SAST; secret scan; dependency review; IaC scan; license policy; test XSS/authz/webhook cases.	Block new high/critical risk or secret. Time-bound, approved exception must name owner and expiry.
Merge to main	Repeat deterministic verification; create version; build frontend/API/IaC/migration bundle once; generate SBOM, checksums and provenance; store immutable artifact keyed by commit SHA.	Only an artifact from protected main proceeds. No environment-specific rebuild.
Deploy staging	Exchange GitHub OIDC through Google Workload Identity Federation; apply reviewed rules/indexes/IaC to the staging project; deploy the API image as a no-traffic Cloud Run revision, then Hosting preview/staging release; seed non-sensitive data.	Health, smoke, API contract, payment sandbox and DAST pass. Deployment is single-concurrency and uses no service-account JSON key.
Approve production	GitHub production environment restricts source branch/tag and requires a reviewer who did not initiate the release; environment variables are scoped.	Approver sees SHA, change summary, test/security results, migration and rollback plan.
Deploy production	Promote the exact API image digest and static web artifact to the production project; deploy a no-traffic revision, test its tagged URL, cut Cloud Run traffic, then publish Hosting config/content pinned to that revision.	Rollback Cloud Run traffic and Hosting release on threshold; release record includes digest/SHA, actor, approval, rules checksum and observations.

Stage / trigger	Automated work	Gate and output
Scheduled/operational	Nightly dependency/security scan; periodic backup-restore verification; synthetic booking/payment health; stale secret/access review reminders.	Issues created with severity/owner/SLA; no silent scan failures.

9.1.14.3 Test strategy and quality gates

Table 17. Test strategy and pass rules.

Layer	Examples for this system	When / pass rule
Unit	Pricing rules; state transitions; cancellation/postponement; 50% deposit; travel/extra image calculation; retention dates.	Every PR; deterministic and fast; zero failures.
Component/integration	Firestore transaction/slot claims; conflicting bookings; outbox; role/resource authorization; Firestore and Storage Rules; signed-media URL policy.	Every PR using Firebase Emulator Suite plus isolated provider fakes; zero failures.
Contract	PayFast/Yoco signed webhook; WhatsApp/email provider; social feed; invalid/replayed payloads.	PR for adapter changes and nightly provider sandbox suite.
End-to-end	Register/login, quote, reserve, sandbox deposit, confirmation, photographer dashboard, reschedule and delivery.	Staging; critical journey must pass.
Security	SAST, dependency/secret/IaC scans; DAST; XSS, IDOR, CSRF, upload and auth tests.	PR/staging; no unapproved high/critical findings.
Performance/resilience	Concurrent reservations, portfolio load, queue backlog, external timeout/retry, recovery.	Before major release and quarterly; agreed SLOs and exactly-one booking invariant.

9.1.14.4 Executable workflow and repository contract

Appendix A shows a clear outline of a functional workflow that directly mapped to single-root rater than pseudocode. The production readiness mandates configuring Next.js static export, creating and API/health endpoint , adding Dockerfile, api and firebase.json, setting Firestore/Storage rules and indexes, and implementing .NET/React test with NuGet lock flies. GitHub staging and production environment isolate non-secret project configuration and Workload Identity Federation setting, enabling secure keyless authentication via short-lived Google credentials exchanged GitHub OIDC (Google Cloud, n.d.-b; GitHub, n.d.-b).

9.1.14.5 Deployment, database and rollback procedure

20. Before merge, classify document/index/rules changes. Use expand/contract: add optional versioned fields and indexes first; deploy code that reads old and new documents; backfill with resumable/idempotent jobs; tighten validation and remove legacy fields only later.
21. Deploy to the staging Firebase project using a least-privilege deployment identity. Test Security Rules in the emulator and against seeded staging data. The Cloud Run runtime identity must not administer IAM, Firebase rules or indexes.
22. Run health, smoke, contract and synthetic-booking checks. Capture deployment evidence and request production approval with the rollback decision point.
23. Deploy the immutable image digest as a no-traffic Cloud Run revision, verify its tagged URL, then move traffic and publish the matching Firebase Hosting release. Observe error rate, latency, task age, Firestore contention/denials, failed webhooks and booking/payment consistency.
24. On application regression, return Cloud Run traffic and Hosting to the previous known-good revision/release. On data incompatibility, prefer forward correction; use Firestore PITR clone/restore only under the rehearsed runbook because reconciliation must protect newer legitimate bookings/payments.
25. After release, record outcome, update the change log and close only when the observation window is healthy and any migration/backfill is complete.

9.1.14.6 DevOps metrics and ownership

Table 18. DevOps metrics and targets.

Metric	Target / signal	Response
Deployment frequency	Small, low-risk releases as work completes; at least weekly during active delivery.	Investigate queue/approval bottleneck, not developer activity alone.
Lead time for change	Median PR-to-production trend decreases without quality regression.	Improve test speed, reusable workflow caching and review sizing.
Change failure rate	<15% releases require rollback/hotfix; trend is more important than a single period.	Regression test/control and Post-incident review and
Mean time to restore	Service-restoring action within 60 minutes for common release failure; business RTO remains <=4 h.	Exercise Cloud Run traffic/Hosting rollback and Firestore restore runbooks quarterly.
Pipeline health	Required checks reliable; flaky-test rate <2%; failed security scans never silently ignored.	Quarantine/fix flaky test; block if a gate cannot produce trustworthy evidence.

Appendix A - Executable GitHub Actions workflow

Page 44

9.1.15 Running Costs

Assumptions

- Hosting: Azure or Google Cloud Platform (GCP) for ASP.NET Core hosting
- Database: Firebase Firestore (serverless, pay-as-you-go) for structured data
- File Storage: Firebase Cloud Storage for images (gallery, profiles, services) and PDF invoices
- Authentication: Firebase Authentication
- Email Service: SendGrid or similar for email notifications
- SMS/WhatsApp: Twilio for WhatsApp notifications
- Currency: All amounts in South African Rand (ZAR)

Cost Projections at different scales

Table 19. Estimated running costs by user scale.

Service/Component	Free Tier	100 Users	1 000 Users	10 000 Users
Firebase Firestore	1 GB storage, 50k reads/day	R0.00	R0.00	R850/mo
Firebase Cloud Storage	5 GB storage, 1 GB/day download	R0.00	R0.00	R450/mo
Firebase Authentication	50K MAUs	R0.00	R0.00	R850/mo
Web Hosting (Azure/GCP)	Free tier (F1/B1)	R0.00	R250/mo	R750/mo
Email Service (SendGrid)	100 emails/day	R0.00	R450/mo	R850/mo
WhatsApp Business API	1 000 conversations/mo	R0.00	R0.00	R650/mo
Domain and SSL	-	R150/year	R150/year	R150/year
Total Monthly	-	R0.00	R850	R4 600
Total Yearly	-	R12.50	R10 350	R55 350

9.1.15.1 Two-year Projection

Based on client input (15 bookings/week current, growing to 50+ bookings/week within 2 years):

Table 20. Two-year running-cost projection.

Year	Users	Monthly Cost	Annual Cost	Cumulative costs
Year 1 – Month 1-6	0-100 users (testing/launch)	R0.00	R0.00	R0.00
Year 1 – Month 7-12	100 – 200 users	R250	R1 500	R1 500
Year 2 – Month 1-6	200 – 500 users	R850	R5 100	R6 600
Year 2 – Month 7-12	500 – 1000 users	R1 250	R7 500	R14 100
Total 2 Years	-	-	**R14 100**	**R14 100**

The cost model scales primarily with Firestore operations, stored/served photographs, Cloud Run compute, outbound messaging and monitoring retention. Controls include thumbnailing and CDN caching, lifecycle deletion after the agreed retention period, Firestore query/index review, budget alerts at 50/80/100%, per-environment quotas and a monthly cost-per-confirmed-booking review.

Cost Breakdown (10 000 Users Scale)

Table 21. Recurring cost components.

Component	Monthly Cost (ZAR)	Annual Cost (ZAR)
Firebase Firestore	R850	R10 200
Firebase Cloud Storage	R450	R5 400
Firebase Authentication	R850	R10 200
Web Hosting	R750	R9 000
Email Service	R850	R10 200
WhatsApp API	R650	R7 800
Total	**R4 600**	**R55 200**

Estimated Storage Requirements

Table 22. Projected storage growth.

Content Type	Average Size	Monthly Growth (1 000 users)	Annual Growth
Gallery Images	5 MB/image	500 images (2.5 GB)	30 GB
Invoice PDFs	200 KB/invoice	50 invoices (10 MB)	120 MB
Profile Pictures	200 KB/user	200 images (40 MB)	480 MB
Service Images	1 MB/image	10 images (10 MB)	120 MB
Total	-	**~2.56 GB/month**	**~30.72 GB/year**

9.1.16 Change Management

The transition from the Union Lab's existing booking method (WhatsApp and Direct Message) to an automated booking platform is an important operational change for both the organization and its users. The change management approach has been done based on established frameworks to make sure that the process goes smoothly (ClickUp, 2026) (Vitally, 2026).

How and why the organization will adopt

Why the organization should adopt:

Human error elimination: Present manual system leads to double booking, forgotten bookings, and miscalculation (rgp, 2026).

Efficiency: Automatic calculations, billing, and deadlines save photographer time.

Scalability: The system helps the organization grow from 15 to 50+ bookings per week.

Adoption Strategy

ADKAR Framework Application

The following ADKAR (Awareness, Desire, Knowledge, Ability, Reinforcement) model will guide the adoption strategy (ClickUp, 2026) (Vitally, 2026)

Table 23. Change-management activities and timeline.

Phase	Activity	Timeline
Awareness	Announce the new system, communicate benefits	Week 1-2
Desire	Showcase quick wins, address concerns	Week 2-3
Knowledge	Deliver comprehensive training	Week 3-4
Ability	Supervised hands on practice during UAT	Week 4-6
Reinforcement	Feedback loops, ongoing support, celebrations	Week 6+

Communication Plan

Table 24. Stakeholder communication plan.

Stakeholder	Message	Channel	Frequency
Owner	System benefits, ROI, progress updates	In-person meetings	Weekly
Photographers	How system helps them, training schedules	WhatsApp group, email	Bi-weekly
Clients (customers)	New booking portal, ease of us	Email, Instagram posts	Once at launch

Handover Plan

Transition Timeline

Table 25. Change implementation roadmap.

Phase	Activity	Duration	Responsibility
Phase 1: Preparation	System setup. Data migration planning. Training material creation	2 weeks	Development Team
Phase 2: Pilot	Pilot with 2 photographs. Collect feedback. Refine system.	2 weeks	Pilot Group
Phase 3: Training	Full team training sessions. Role based workshops (admin, photog., client).	1 week	Change Manager

	Create user guides.		
Phase 4: G0-Live	System Launch. 24/7 support for first week. Monitor adoption metrics.	1 week	All Team
Phase 5: Stabilisation	Weekly adoption reviews. Adress issues. Gather feedback.	30-60-90 days (HubSpot, 2026)	Change Manager

Training Plan

Table 26. Training plan.

Role	Training Content	Method	Duration
photographers	Dashboard usage, viewing bookings, updating status, payment tracking	Hands-on workshop + User guide	3 hours
Admin staff	Managing bookings, photographers, reports, system configuration	Workshop + Administrator Guide	4 hours
Clients (customers)	Booking process, viewing services, making payments	Video tutorial + FAQs	15 minutes

Support Strategy

Table 27. Support plan.

Support Type	Description	Availability
Live Chat	In-app support chat	9AM – 5PM weekdays
Email Support	support@theunionlab.co.za	24-hour response
WhatsApp	Dedicated support WhatsApp line	9AM – 5PM weekdays
FAQ/KB	Self-service knowledge base	24/7

Measuring Adoption Success

Table 28. Change-management success metrics.

Metric	Target	Measurement Method
Photographer Dashboard Login Rate	90% daily	Firebase Analytics

Bookings Made Through System	95% within 1 month	Database Query
Successful Payment Rate	80% deposit within 2 days	Payment Logs
User Satisfaction	4.5/5 average rating	Feedback Surveys
Support Ticket Volume	<10 per week	Ticketing System

Risk Mitigation

Table 29. Change-management risks and mitigations.

Risk	Impact	Mitigation
Resistance to change	High	Identify champions, communication benefits, involve users in design
Technical issues during transition	Medium	Phased rollout, through testing, rollback plan
Inadequate training	Medium	Multiple training formats, ongoing support, refresher sessions.
Data migration errors	High	Data validation scripts, backup Strategy, reconciliation process (cleverance, 2026).

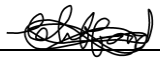
.

9.1.17 Appendices

9.1.17.1 Declaration of Authenticity

We declare that this submission is the group's own work, that all external ideas, documents and technical sources have been acknowledged, and that every member's contribution has been reviewed for consistency before submission. Each student remains responsible for compliance with the institution's academic-integrity and permitted-tool rules.

Table 30. Declaration of Authenticity signatures.

Student	Student number	Signature	Date
Clifford Naude	ST10258513		2026/08/12
Mikhaeel Ismail	ST10433968		2026/08/12
Siyabonga Nkomazana	ST10444685		2026/08/12
Atlegang Seboge	ST10443510		2026/08/12
Ruan Hartman	ST10453370		2026/0812
Ntokozo Khumalo	ST10447179		2026/08/12

Appendix A-Executable GitHub Actions Workflow

```
name: photography-platform-ci-cd

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
```

```
env:
  DOTNET_VERSION: 10.0.x
  NODE_VERSION: 20
  API_PROJECT: CB Photography Webapp.csproj

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  verify:
    name: Build, test and validate Firebase policy
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: ${{ env.DOTNET_VERSION }}
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm

      - name: Restore, format, build and test API
        run: |
          dotnet restore "${API_PROJECT}"
          dotnet format "${API_PROJECT}" --verify-no-changes --no-restore
          dotnet build "${API_PROJECT}" -c Release --no-restore
          dotnet test "${API_PROJECT}" -c Release --no-build            --logger "trx;LogFileName=dotnet-
tests.trx"

      - name: Restore, lint, type-check, test and export web
        run: |
          npm ci
          npm run lint
          npx tsc --noEmit
          npm run test --if-present
          npm run build
          test -d out

      - name: Test Firestore and Storage Rules in emulators
        run: |
          test -f firebase.json
          test -f firestore.rules
          test -f storage.rules
          npx firebase-tools@latest emulators:exec             --only firestore,storage "npm run
test:firebase"

      - name: Upload test evidence
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-evidence-${{ github.sha }}
          path: |
            **/TestResults/**
            coverage/**
          if-no-files-found: warn
          retention-days: 30

  codeql:
    name: CodeQL analysis
    runs-on: ubuntu-latest
    permissions:
      contents: read
```

```
    security-events: write
    packages: read
  steps:
    - uses: actions/checkout@v4
    - uses: github/codeql-action/init@v3
      with:
        languages: csharp,javascript-typescript
    - uses: github/codeql-action/autobuild@v3
    - uses: github/codeql-action/analyze@v3

dependency-review:
  name: Dependency review
  if: github.event_name == 'pull_request'
  runs-on: ubuntu-latest
  permissions:
    contents: read
  steps:
    - uses: actions/checkout@v4
    - uses: actions/dependency-review-action@v4
      with:
        fail-on-severity: high
        deny-licenses: GPL-3.0, AGPL-3.0

package:
  name: Build immutable release artifact
  if: github.event_name == 'push'
  needs: [verify, codeql]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ env.NODE_VERSION }}
        cache: npm
    - name: Build static web once
      run: |
        npm ci
        npm run build
        test -d out
    - name: Build API container once
      run: |
        test -f Dockerfile.api
        docker build --file Dockerfile.api             --tag photography-api:${GITHUB_SHA} .
        docker save photography-api:${GITHUB_SHA}             --output api-image.tar
    - name: Assemble release and checksums
      run: |
        mkdir release
        cp -R out release/web
        cp api-image.tar firebase.json firestore.rules             storage.rules release/
        if [ -f firestore.indexes.json ]; then             cp firestore.indexes.json release/; fi
        find release -type f -print0 | sort -z |             xargs -0 sha256sum > release/SHA256SUMS.txt
    - uses: actions/upload-artifact@v4
      with:
        name: release-${{ github.sha }}
        path: release/
        if-no-files-found: error
        retention-days: 90

deploy-staging:
  name: Deploy and test staging project
  if: github.event_name == 'push'
  needs: package
  runs-on: ubuntu-latest
  environment: staging
```

```
    permissions:
      contents: read
      id-token: write
    concurrency: staging
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: release-${{ github.sha }}
          path: release
      - uses: google-github-actions/auth@v3
        with:
          workload_identity_provider: ${{ vars.WIF_PROVIDER }}
          service_account: ${{ vars.DEPLOY_SERVICE_ACCOUNT }}
      - uses: google-github-actions/setup-gcloud@v3
      - name: Push exact API image to shared Artifact Registry
        run: |
          IMAGE="${{ vars.ARTIFACT_REGION }}-docker.pkg.dev/${{ vars.ARTIFACT_PROJECT_ID }}/${{
vars.ARTIFACT_REPOSITORY }}/photography-api:${GITHUB_SHA}"
          gcloud auth configure-docker             "${{ vars.ARTIFACT_REGION }}-docker.pkg.dev" --quiet
          docker load --input release/api-image.tar
          docker tag photography-api:${GITHUB_SHA} "$IMAGE"
          docker push "$IMAGE"
          echo "IMAGE=$IMAGE" >> "$GITHUB_ENV"
      - name: Deploy staging API
        run: |
          gcloud run deploy "${{ vars.CLOUD_RUN_SERVICE }}"             --project "${{
vars.FIREBASE_PROJECT_ID }}"             --region "${{ vars.CLOUD_RUN_REGION }}"             --image
"$IMAGE"             --service-account "${{ vars.RUNTIME_SERVICE_ACCOUNT }}"             --set-env-vars
"GOOGLE_CLOUD_PROJECT=${{ vars.FIREBASE_PROJECT_ID }}"             --quiet
      - name: Deploy rules, indexes and Hosting to staging
        run: |
          cp -R release/web out
          cp release/firebase.json release/firestore.rules             release/storage.rules .
          if [ -f release/firestore.indexes.json ]; then             cp release/firestore.indexes.json .;
fi
          npx firebase-tools@latest deploy             --only firestore,storage,hosting             --
project "${{ vars.FIREBASE_PROJECT_ID }}"             --non-interactive
      - name: Smoke and DAST staging
        run: |
          curl --fail --retry 12 --retry-delay 10             "${{ vars.STAGING_API_URL }}/health"
          curl --fail --retry 12 --retry-delay 10             "${{ vars.STAGING_WEB_URL }}/"
          docker run --rm -v "$PWD:/zap/wrk/:rw" -t             ghcr.io/zaproxy/zaproxy:stable zap-
baseline.py             -t "${{ vars.STAGING_WEB_URL }}" -r zap-report.html -I

  deploy-production:
    name: Approve and release production project
    if: github.event_name == 'push'
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    permissions:
      contents: read
      id-token: write
    concurrency: production
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: release-${{ github.sha }}
          path: release
      - uses: google-github-actions/auth@v3
        with:
          workload_identity_provider: ${{ vars.WIF_PROVIDER }}
          service_account: ${{ vars.DEPLOY_SERVICE_ACCOUNT }}
      - uses: google-github-actions/setup-gcloud@v3
```

```
      - name: Deploy exact image as no-traffic revision
        shell: bash
        run: |
          IMAGE="${{ vars.ARTIFACT_REGION }}-docker.pkg.dev/${{ vars.ARTIFACT_PROJECT_ID }}/${{
vars.ARTIFACT_REPOSITORY }}/photography-api:${GITHUB_SHA}"
          OLD_REVISION=$(gcloud run services describe             "${{ vars.CLOUD_RUN_SERVICE }}"
--project "${{ vars.FIREBASE_PROJECT_ID }}"             --region "${{ vars.CLOUD_RUN_REGION }}"
--format='value(status.traffic[0].revisionName)')
          gcloud run deploy "${{ vars.CLOUD_RUN_SERVICE }}"             --project "${{
vars.FIREBASE_PROJECT_ID }}"             --region "${{ vars.CLOUD_RUN_REGION }}"             --image
"$IMAGE" --no-traffic             --tag "candidate-${GITHUB_SHA:0:8}"             --service-account "${{
vars.RUNTIME_SERVICE_ACCOUNT }}"             --set-env-vars "GOOGLE_CLOUD_PROJECT=${{
vars.FIREBASE_PROJECT_ID }}"             --quiet
          NEW_REVISION=$(gcloud run revisions list             --service "${{ vars.CLOUD_RUN_SERVICE }}"
--project "${{ vars.FIREBASE_PROJECT_ID }}"             --region "${{ vars.CLOUD_RUN_REGION }}"
--sort-by='~metadata.creationTimestamp' --limit=1             --format='value(metadata.name)')
          echo "OLD_REVISION=$OLD_REVISION" >> "$GITHUB_ENV"
          echo "NEW_REVISION=$NEW_REVISION" >> "$GITHUB_ENV"
          CANDIDATE_URL=$(gcloud run services describe             "${{ vars.CLOUD_RUN_SERVICE }}"
--project "${{ vars.FIREBASE_PROJECT_ID }}"             --region "${{ vars.CLOUD_RUN_REGION }}"
--format="value(status.traffic[tag=candidate-${GITHUB_SHA:0:8}].url)")
          curl --fail --retry 12 --retry-delay 10             "${CANDIDATE_URL}/health"
      - name: Cut over API traffic
        run: |
          gcloud run services update-traffic             "${{ vars.CLOUD_RUN_SERVICE }}"             --
project "${{ vars.FIREBASE_PROJECT_ID }}"            --region "${{ vars.CLOUD_RUN_REGION }}"
--to-revisions "${NEW_REVISION}=100" --quiet
      - name: Deploy production rules, indexes and Hosting
        run: |
          cp -R release/web out
          cp release/firebase.json release/firestore.rules            release/storage.rules .
          if [ -f release/firestore.indexes.json ]; then            cp release/firestore.indexes.json .;
fi
          npx firebase-tools@latest deploy             --only firestore,storage,hosting             --
project "${{ vars.FIREBASE_PROJECT_ID }}"            --non-interactive
      - name: Verify production; roll API traffic back on failure
        shell: bash
        run: |
          if ! curl --fail --retry 6 --retry-delay 10              "${{ vars.PRODUCTION_API_URL
}}/health" ||              ! curl --fail --retry 6 --retry-delay 10              "${{
vars.PRODUCTION_WEB_URL }}/"; then
            gcloud run services update-traffic              "${{ vars.CLOUD_RUN_SERVICE }}"
--project "${{ vars.FIREBASE_PROJECT_ID }}"              --region "${{ vars.CLOUD_RUN_REGION }}"
--to-revisions "${OLD_REVISION}=100" --quiet
            echo "API rolled back. Redeploy the retained prior web artifact."
            exit 1
          fi
```

9.1.17.3 Required GitHub Environment Configuration

Table 31. GitHub environment configuration.

Configuration	staging	production
Protection	Protected main branch; automatic after package gate.	Protected main only; required reviewer; prevent self-review; no bypass.
Federated identity variables	WIF_PROVIDER and DEPLOY_SERVICE_ACCOUNT scoped to the staging deployment identity; no JSON key.	Equivalent values scoped to a separate least-privilege production identity.

Configuration	staging	production
Cloud variables	FIREBASE_PROJECT_ID, CLOUD_RUN_SERVICE/REGION, RUNTIME_SERVICE_ACCOUNT, shared ARTIFACT_* values, STAGING_API_URL and STAGING_WEB_URL.	Production project/service/runtime identity, same shared image registry, PRODUCTION_API_URL and PRODUCTION_WEB_URL.
Concurrency	One staging deployment at a time.	One production deployment at a time; pending run does not overtake an approved release.

The listing is executable after the repository-contract files and environment variables are added. Resolve each external action tag and Firebase CLI version to a reviewed immutable version/commit before production. Firestore document evolution uses versioned, backward-compatible fields, reviewed indexes and idempotent backfills; the runtime identity must not administer rules, indexes or IAM. The shared Artifact Registry must grant the production Cloud Run service agent read access so staging and production use the exact same image digest. Firebase Hosting rollback is performed by redeploying the retained prior release artifact (and, when pinTag is configured, its matched Cloud Run revision).

Executable data schema

Booking schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "Id": {
      "type": "string"
    },
    "BookingReference": {
      "type": "string"
    },
    "ServiceId": {
      "type": "string"
    },
    "ServiceName": {
      "type": "string"
    },
    "PackageId": {
      "type": "string"
    },
    "PackageName": {
      "type": "string"
    },
    "PhotographerId": {
      "type": "string"
    },
    "PhotographerName": {
      "type": "string"
    },
    "Date": {
      "type": "string"
    },
    "TimeSlot": {
      "type": "string"
    },
    "BasePriceZAR": {
      "type": "number"
    },
    "TravelFeeZAR": {
      "type": "number"
    },
    "AddonsTotalZAR": {
      "type": "number"
    },
    "SubtotalZAR": {
      "type": "number"
    },
    "DepositRequiredZAR": {
      "type": "number"
    },
    "RemainingBalanceZAR": {
```

```
      "type": "number"
    },
    "Status": {
      "type": "string"
    },
    "PaymentStatus": {
      "type": "string"
    },
    "CreatedAt": {
      "type": "string"
    }
  },
  "required": [
    "Id",
    "BookingReference",
    "ServiceId",
    "ServiceName",
    "PackageId",
    "PackageName",
    "PhotographerId",
    "PhotographerName",
    "Date",
    "TimeSlot",
    "BasePriceZAR",
    "TravelFeeZAR",
    "AddonsTotalZAR",
    "SubtotalZAR",
    "DepositRequiredZAR",
    "RemainingBalanceZAR",
    "Status",
    "PaymentStatus",
    "CreatedAt"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "Id": "Example string",
      "BookingReference": "Example string",
      "ServiceId": "Example string",
      "ServiceName": "Example string",
      "PackageId": "Example string",
      "PackageName": "Example string",
      "PhotographerId": "Example string",
      "PhotographerName": "Example string",
      "Date": "Example string",
      "TimeSlot": "Example string",
      "BasePriceZAR": 123.45,
      "TravelFeeZAR": 123.45,
      "AddonsTotalZAR": 123.45,
      "SubtotalZAR": 123.45,
      "DepositRequiredZAR": 123.45,
      "RemainingBalanceZAR": 123.45,
      "Status": "Example string",
      "PaymentStatus": "Example string",
      "CreatedAt": "Example string"
```

```
  }
 ]
}
```

CustomerInfo schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "FullName": {
      "type": "string"
    },
    "Email": {
      "type": "string"
    },
    "Phone": {
      "type": "string"
    },
    "EventType": {
      "type": "string"
    }
  },
  "required": [
    "FullName",
    "Email",
    "Phone",
    "EventType"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "FullName": "Example string",
      "Email": "Example string",
      "Phone": "Example string",
      "EventType": "Example string"
    }
  ]
}
```

DashboardStats schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "TotalBookings": {
      "type": "integer"
    },
    "GrossRevenueZAR": {
      "type": "number"
    },
    "OutstandingBalancesZAR": {
      "type": "number"
    },
    "PendingEditingCount": {
      "type": "integer"
    },
    "TotalServicesCount": {
      "type": "integer"
    },
    "TotalPhotographersCount": {
      "type": "integer"
    }
  },
  "required": [
    "TotalBookings",
    "GrossRevenueZAR",
    "OutstandingBalancesZAR",
    "PendingEditingCount",
    "TotalServicesCount",
    "TotalPhotographersCount"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "TotalBookings": 42,
      "GrossRevenueZAR": 123.45,
      "OutstandingBalancesZAR": 123.45,
      "PendingEditingCount": 42,
      "TotalServicesCount": 42,
      "TotalPhotographersCount": 42
    }
  ]
}
```

LocationInfo schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "VenueName": {
      "type": "string"
    },
    "Address": {
      "type": "string"
    },
    "Suburb": {
      "type": "string"
    },
    "City": {
      "type": "string"
    },
    "Province": {
      "type": "string"
    },
    "PostalCode": {
      "type": "string"
    },
    "DistanceFromBaseKm": {
      "type": "number"
    },
    "TravelFeeZAR": {
      "type": "number"
    }
  },
  "required": [
    "VenueName",
    "Address",
    "Suburb",
    "City",
    "Province",
    "PostalCode",
    "DistanceFromBaseKm",
    "TravelFeeZAR"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "VenueName": "Example string",
      "Address": "Example string",
      "Suburb": "Example string",
      "City": "Example string",
      "Province": "Example string",
      "PostalCode": "Example string",
      "DistanceFromBaseKm": 123.45,
      "TravelFeeZAR": 123.45
    }
  ]
}
```

PackageOption schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "Id": {
      "type": "string"
    },
    "ServiceId": {
      "type": "string"
    },
    "Name": {
      "type": "string"
    },
    "StartingPriceZAR": {
      "type": "number"
    },
    "DurationHours": {
      "type": "integer"
    },
    "Features": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 0
    },
    "IsPopular": {
      "type": "boolean"
    }
  },
  "required": [
    "Id",
    "ServiceId",
    "Name",
    "StartingPriceZAR",
    "DurationHours",
    "Features",
    "IsPopular"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "Id": "Example string",
      "ServiceId": "Example string",
      "Name": "Example string",
      "StartingPriceZAR": 123.45,
      "DurationHours": 42,
      "Features": [
        "Example string",
        "Example string"
      ],
      "IsPopular": true
    }
  ]
}
```

Photographer schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "Id": {
      "type": "string"
    },
    "Name": {
      "type": "string"
    },
    "Role": {
      "type": "string"
    },
    "Email": {
      "type": "string"
    },
    "Phone": {
      "type": "string"
    },
    "AvatarUrl": {
      "type": "string"
    },
    "Bio": {
      "type": "string"
    },
    "Specialties": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 0
    },
    "AvailableDays": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 0
    },
    "CompletedShoots": {
      "type": "integer"
    }
  },
  "required": [
    "Id",
    "Name",
    "Role",
    "Email",
    "Phone",
    "AvatarUrl",
    "Bio",
    "Specialties",
```

```
    "AvailableDays",
    "CompletedShoots"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "Id": "Example string",
      "Name": "Example string",
      "Role": "Example string",
      "Email": "Example string",
      "Phone": "Example string",
      "AvatarUrl": "Example string",
      "Bio": "Example string",
      "Specialties": [
        "Example string",
        "Example string"
      ],
      "AvailableDays": [
        "Example string",
        "Example string"
      ],
      "CompletedShoots": 42
    }
  ]
}
```


Service schema


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "title": "Generated Schema",
  "description": "Auto-generated JSON Schema",
  "properties": {
    "Id": {
      "type": "string"
    },
    "Title": {
      "type": "string"
    },
    "Slug": {
      "type": "string"
    },
    "Subtitle": {
      "type": "string"
    },
    "Description": {
      "type": "string"
    },
    "StartingPriceZAR": {
      "type": "number"
    },
    "HeroImageUrl": {
      "type": "string"
    },
    "Packages": {
      "type": "array",
      "items": {
```

```
        "type": "null"
      },
      "minItems": 0
    }
  },
  "required": [
    "Id",
    "Title",
    "Slug",
    "Subtitle",
    "Description",
    "StartingPriceZAR",
    "HeroImageUrl",
    "Packages"
  ],
  "additionalProperties": false,
  "examples": [
    {
      "Id": "Example string",
      "Title": "Example string",
      "Slug": "Example string",
      "Subtitle": "Example string",
      "Description": "Example string",
      "StartingPriceZAR": 123.45,
      "HeroImageUrl": "Example string",
      "Packages": [
        null,
        null
      ]
    }
  ]
}
```

References

Business and assessment sources

- Camryn Burger Photography (n.d.) Terms & Conditions. Unpublished client document supplied to the project team.
- Independent Institute of Education (IIE) (2026) INSY7315 WIL Task 1 Documentation and Marking Rubric. Unpublished module document.
- INSY7315 WIL Group (2026a) Proposal Presentation: The Union Lab & Camryn Burger Photography. Unpublished group presentation.
- INSY7315 WIL Group (2026b) CB Photography Webapp: ASP.NET Core and Next.js prototype. Unpublished source-code archive supplied by the project team on 12 August 2026.
- The Union Lab Wedding Photography & Camryn Burger Photography (n.d.) Business Description and From the Start to End. Unpublished client briefing documents (WIL_INSY and WIL_INSY2).

Technical sources

- GitHub (n.d.-a) Deployments and environments. Available at: https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments (Accessed: 10 August 2026).
- GitHub (n.d.-b) OpenID Connect reference. Available at: https://docs.github.com/en/actions/reference/security/oidc (Accessed: 10 August 2026).
- GitHub (n.d.-c) Workflow syntax for GitHub Actions. Available at: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax (Accessed: 10 August 2026).
- GitHub (n.d.-d) Configuring the dependency review action. Available at: https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/manage-your-dependency-security/configure-dependency-review-action (Accessed: 10 August 2026).
- Firebase (n.d.-a) Cloud Firestore data model. Available at: https://firebase.google.com/docs/firestore/data-model (Accessed: 12 August 2026).
- Firebase (n.d.-b) Transactions and batched writes. Available at: https://firebase.google.com/docs/firestore/manage-data/transactions (Accessed: 12 August 2026).
- Firebase (n.d.-c) Cloud Firestore locations. Available at: https://firebase.google.com/docs/firestore/locations (Accessed: 12 August 2026).
- Firebase (n.d.-d) Serve dynamic content and microservices with Cloud Run. Available at: https://firebase.google.com/docs/hosting/cloud-run (Accessed: 12 August 2026).
- Firebase (n.d.-e) Verify ID tokens. Available at: https://firebase.google.com/docs/auth/admin/verify-id-tokens (Accessed: 12 August 2026).
- Firebase (n.d.-f) Verify App Check tokens from a custom backend. Available at: https://firebase.google.com/docs/app-check/custom-resource-backend (Accessed: 12 August 2026).
- Firebase (n.d.-g) Understand Firebase Security Rules for Cloud Storage. Available at: https://firebase.google.com/docs/storage/security (Accessed: 12 August 2026).
- Firebase (n.d.-h) Fix insecure Cloud Firestore Security Rules. Available at: https://firebase.google.com/docs/firestore/security/insecure-rules (Accessed: 12 August 2026).
- Firebase (n.d.-i) Work with point-in-time recovery. Available at: https://firebase.google.com/docs/firestore/use-pitr (Accessed: 12 August 2026).
- Firebase (n.d.-j) Back up and restore data. Available at: https://firebase.google.com/docs/firestore/backups (Accessed: 12 August 2026).
- Google Cloud (n.d.-a) Deploy container images to Cloud Run services. Available at: https://cloud.google.com/run/docs/deploying (Accessed: 12 August 2026).
- Google Cloud (n.d.-b) Configure Workload Identity Federation with deployment pipelines. Available at: https://cloud.google.com/iam/docs/workload-identity-federation-with-deployment-pipelines (Accessed: 12 August 2026).
- OWASP (n.d.-a) Cross Site Scripting Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html (Accessed: 10 August 2026).

- OWASP (n.d.-b) Application Security Verification Standard. Available at: https://owasp.org/www-project-application-security-verification-standard/ (Accessed: 10 August 2026).
- Kalbach, J., 2016. Mapping Experiences: Aguide to creating value through journeys, blueprints and diagrams. Sebastopol, CA: O'Reilley Media.
- Project Management Institute, 2021. A guide to the project management body of knowledge (PMBOK Guide) and the Standard for project management. 7th ed. Newtown Square, PA: Project Management Institute.
- Sommerville, I., 2016. *Software Engineering.* 10th ed. Harlow: Pearson.
- Geeks For Geeks, 2026. UML Class Diagram, 13 May 2026. [Online]. Available at:

  https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/
  [Accessed 13/08/2026]

- Linnaeus University, n.d. UML Domain Model, n.d. [Online]. Available at:

  https://coursepress.lnu.se/courses/object-oriented-analysis-and-design/02-theory/domain_class_diagram
  [Accessed 13/08/2026]

- Geeks For Geeks, 2026. Sequence Diagrams – Unified Modeling Language (UML), 27 June 2026. [Online]. Available at:

  https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/
  [Accessed 13/08/2026]

- Geeks For Geeks, 2026. State Machine Diagrams | Unified Modeling Language (UML), 1 June 2026. [Online]. Available at:

  https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-state-diagrams/
  [Accessed 13/08/2026]

-

I declare that I used an Artificial Intelligence (AI) tool solely to assist with the **formatting and presentation** of this work. The AI tool was used to improve the structure.